

most-exchange

Operator's Manual

Installing, configuring and operating a deterministic low-latency matching engine: the shard and its five processes, the control plane that authors what they boot from, and the interfaces an operator drives them through.

Revision 2026-09-06

Applies to the current implementation, at roughly 80% of the specified feature set. Sections marked NOT IMPLEMENTED describe behaviour that is designed but not built.

Contents

1 Introduction

- 1.1 What this system is
- 1.2 The processes
- 1.3 Two boundaries, both binary
- 1.4 The two kinds of configuration
- 1.5 Prices, quantities and identifiers
- 1.6 Session phases
- 1.7 What is not built
- 1.8 How to read this manual

2 Development deployment and operation

- 2.1 Prerequisites
- 2.2 Build
- 2.3 The whole stack at once
- 2.4 Verifying the stack came up
- 2.5 The operator CLI
- 2.6 Driving a trade
- 2.7 Watching the book
- 2.8 The admin UI
- 2.9 What is development-only
- 2.10 Running without Docker
- 2.11 Automated end-to-end checks
- 2.12 Shutting down

3 Production deployment

- 3.1 The rule that shapes everything
- 3.2 Topology
- 3.3 Machine preparation
- 3.4 Core allocation
- 3.5 Aeron buffer sizing
- 3.6 Cluster configuration
- 3.7 Market data: multicast
- 3.8 Process management
- 3.9 Bringing a shard up
- 3.10 Blocking gaps

4 Configuration in full

- 4.1 How a process is configured
- 4.2 The shard security file
- 4.3 The participant registry

- 4.4 Engine
 - 4.5 Gateway
 - 4.6 Market data
 - 4.7 Discovery
 - 4.8 The cluster host
 - 4.9 The control plane
 - 4.10 Authoring configuration in the control plane
 - 4.11 Publishing a release
-

5 **Daily operations, order flow and monitoring**

- 5.1 The trading day
 - 5.2 The trading calendar
 - 5.3 Operator commands
 - 5.4 Order flow
 - 5.5 Market data operations
 - 5.6 Snapshots and restarts
 - 5.7 Measuring
 - 5.8 Monitoring
-

6 **Troubleshooting and maintenance**

- 6.1 First moves
 - 6.2 Startup failures
 - 6.3 The engine refuses to restore its snapshot
 - 6.4 Recovering a halted security
 - 6.5 Order flow problems
 - 6.6 Market data problems
 - 6.7 Maintenance procedures
 - 6.8 Reading the logs
 - 6.9 Consolidated list of what is not implemented
-

7 **Index**

- 7.1 Terms and concepts
- 7.2 Commands
- 7.3 Configuration keys and environment variables
- 7.4 Further reading

1. Introduction

1.1 What this system is

most-exchange is a matching engine for continuously traded, order-driven markets. It runs as one or more **shards**; each shard hosts up to ten securities, holds an independent limit order book for each of them, and is replicated across an odd number of machines by Aeron Cluster's Raft implementation.

A shard is a **deterministic single-threaded state machine**. One thread consumes a replicated log of commands and applies them in order; every node runs the identical code over the identical log and must arrive at byte-identical state. Everything an operator can do to a running market is expressed as a message in that log, and nothing an operator does is applied on one node and not another.

The design targets 100,000 orders per second per security across ten securities on a shard — a one-microsecond aggregate budget per order. The engine's own share of that, measured on a development machine, is 0.42 µs at the median for a whole message.

SHARD

The unit of replication and of scale. One shard is one Raft group, one replicated log, one matching thread, and at most ten securities with fully independent books. There is no cross-instrument matching, so capacity is added by running more shards, never by adding securities to one.

DETERMINISM

The property that the same log produces the same state on every node. It is why wall-clock reads, unbounded allocation and unvalidated array indexing are prohibited on the engine's path: a crash or a divergence does not affect one node, it affects every node at the same log position simultaneously.

1.2 The processes

A shard is five processes plus a control plane. They are separate processes on purpose — the media driver allocates memory and the engine must not, market data fan-out must never touch the matching thread, and the gateway is restarted on a different schedule from the Raft group.

Process	Binary	Role
Cluster host	most-cluster	Aeron media driver, archive and consensus module in one process. Owns the replicated log, and authenticates connecting gateways.
Engine	engine	The Aeron ClusteredService — books, matching, auctions, snapshots. Attaches to a consensus module that must already exist.
Gateway	gateway	Order entry. Validates securityId, forwards to the cluster, translates execution reports back out. Holds no per-order state.
Market data	market-data	Consumes the engine's book event stream and derives L1, L2 and L3 plus an L2 recovery snapshot feed.
Discovery	discovery	Broadcasts the tradable universe — symbol to shard to gateway endpoints — on a repeating cycle.

Process	Binary	Role
Control plane	control	Authors reference data in Postgres, publishes immutable release artifacts, drives operator commands, runs the trading calendar, and serves the admin UI.

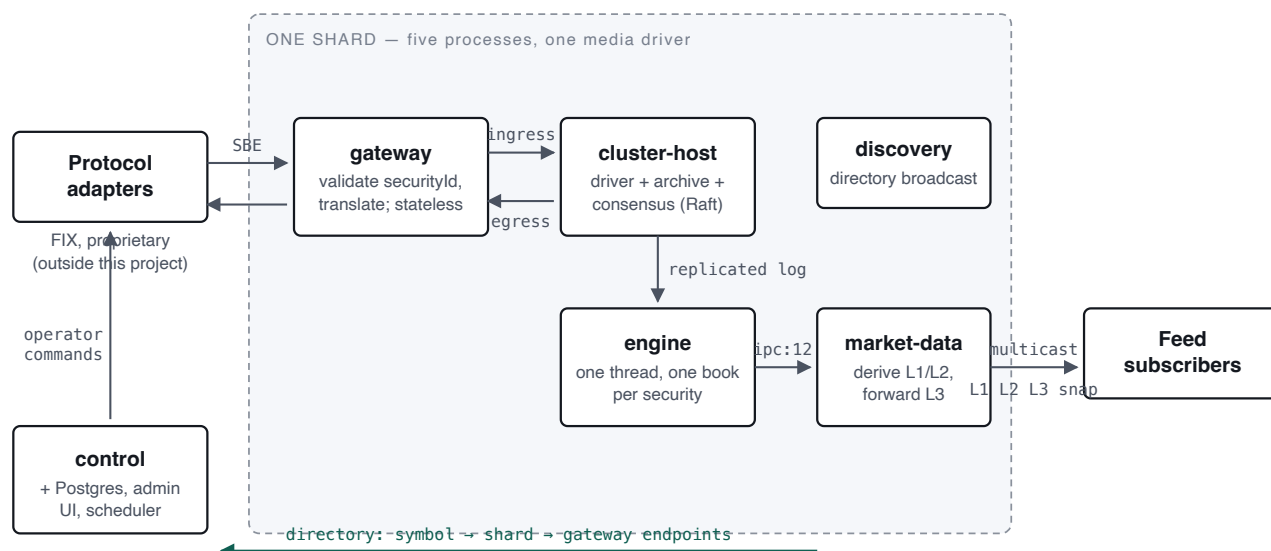


Figure 1.1 The five shard processes, the control plane, and what flows between them.

Only the control plane is authenticated, and the reason is worth stating plainly: it is the only process that *decides* something. The other five apply a replicated log or forward bytes.

1.3 Two boundaries, both binary

Order entry and market data are both **SBE over Aeron**. There is no FIX anywhere in this project. Protocol adapters that speak FIX or a proprietary session protocol sit upstream of the gateway and downstream of market data, and are outside this codebase.

The wire schema is `sbe/src/main/resources/message-schema.xml`; codecs are generated from it and byte offsets are never written by hand.

SBE

Simple Binary Encoding — a fixed-layout binary message format with generated codecs. Messages are encoded directly into the Aeron log buffer through `tryClaim`, so publishing a message copies nothing.

1.4 The two kinds of configuration

Everything an operator sets falls into exactly one of two categories, and confusing them is the most common source of a shard that starts but is subtly wrong.

Geometry and topology — which securities exist, on which shard, with what tick size, price floor, ladder size and order capacity; which gateway speaks for which participant; where the gateway's client endpoints are. This is

authored in the control plane, published as an immutable numbered **release**, and read from a *file* at boot by every process that needs it. Every process prints a `fingerprint()` of what it read at startup, and those fingerprints must agree.

Node-local settings — Aeron directories, cluster directories, feed channels, metrics switches, thread and idle settings. These live in each process’s own properties file and are templated per host. They are deliberately not in the database and not in a release.

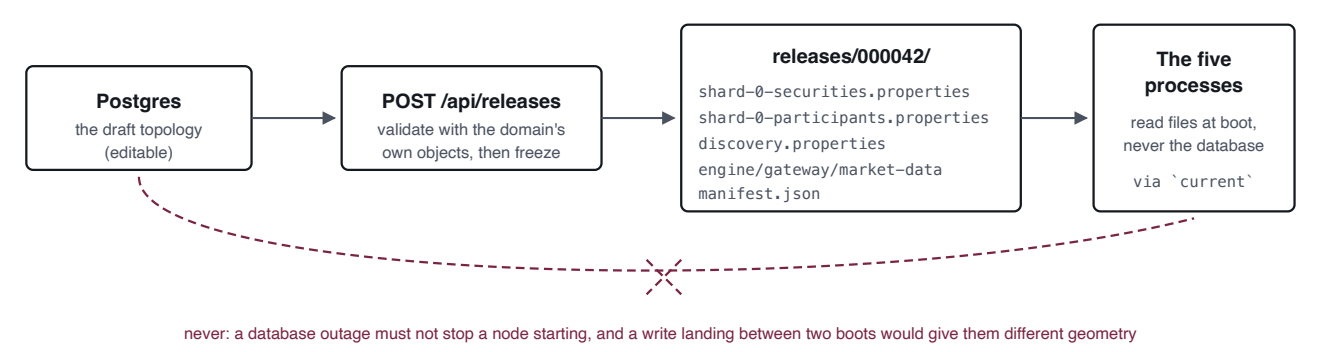


Figure 1.2 A release is authored in Postgres, frozen into files, and read from files. The database is never on a boot path.

FINGERPRINT

A short hash over a shard’s published geometry, printed by the engine, gateway, market-data and discovery processes at startup. Three nodes printing three different values is a divergence waiting to happen on the first order, and consensus cannot catch it — this is the one misconfiguration Raft will not save you from.

RELEASE

An immutable, numbered directory of rendered configuration artifacts produced by `POST /api/releases`. Republishing never overwrites; it allocates the next version. Deployments point a `current` symlink at a release directory, and processes read through that symlink.

1.5 Prices, quantities and identifiers

Prices and quantities are fixed-point signed 64-bit integers with **eight implied decimals**. There is no floating point anywhere in pricing or matching arithmetic, and no operator-facing interface should introduce one.

Displayed	On the wire
100.00	10000000000
0.01 (a tick)	1000000
101.30	10130000000

`securityId` is a 32-bit integer and is the primary identity of an instrument everywhere on the wire; symbols exist for humans and for routing lookups. `participantId` identifies the trading firm. `exchangeOrderId` is allocated by the engine and is what a cancel refers to.

1.6 Session phases

Every security is in one of four phases. Orders are **accepted and booked in every phase except CLOSED**; only *matching* is gated on **CONTINUOUS**. A crossed book outside continuous trading is therefore normal, and is resolved by the uncrossing algorithm.

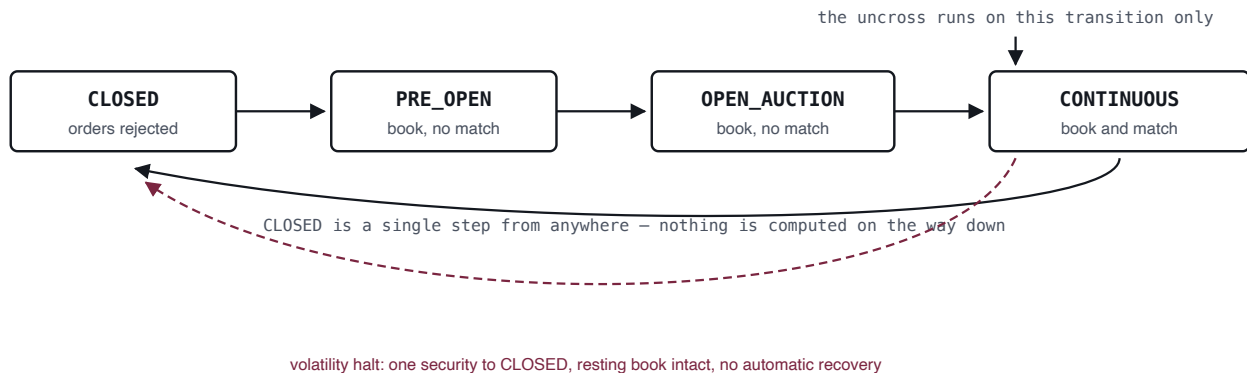


Figure 1.3 The session lifecycle. Phases are walked, not jumped: the uncross runs on exactly one transition.

The transition that matters is `OPEN_AUCTION → CONTINUOUS`, which runs the uncross. Sending `CONTINUOUS` from any other phase is accepted and silently skips the auction, leaving a crossed resting book crossed until an aggressor happens to arrive.

A **session transition is shard-wide**. There is no per-security session command, so moving one security's phase moves every book on that shard.

1.7 What is not built

The manual marks unimplemented behaviour inline, in callouts like the one below, at the point where an operator would otherwise expect it to work. The consolidated list is in 6.9.

ORDER-ENTRY AUTHORIZATION

A gateway's participant claims decide **where a participant's execution reports are delivered**, not whether it may trade. `UNAUTHORIZED_PARTICIPANT` exists in the wire schema and is raised by nothing, so a gateway can currently submit orders for a participant it does not claim.

1.8 How to read this manual

Section 2 brings the whole system up on one machine and is the fastest way to acquire the vocabulary the rest of the manual uses. Section 3 is the production topology. Section 4 is the complete configuration reference and is meant to be consulted rather than read. Section 5 is the daily operational procedure — order flow, market data, the trading day, and what to measure. Section 6 is diagnosis and maintenance. Section 7 indexes the terms and commands by section.

Command transcripts in this manual are real output from a running system, not illustrations.

Companion documents in the repository, which this manual summarises rather than replaces: `docs/Design.md` is the authoritative specification, `docs/ControlPlane.md` covers the control plane in depth,

`docs/ProdDeployment.md` the production tier, `docs/LocalTesting.md` the manual walkthrough, and `deploy/README.md` the development stack.

2. Development deployment and operation

This section brings up the complete system — control plane, database, admin UI and one full shard — on a single machine, and drives a trade through it. Everything in later sections assumes the vocabulary established here.

A DEVELOPMENT STACK IS NOT A SMALL PRODUCTION STACK

The stack in `deploy/` runs **one** cluster node, a shared secret committed to the repository, a fixed admin password, and dynamic MDC in place of multicast. Each of those is listed in 2.9 with what to do instead. Do not carry them forward.

2.1 Prerequisites

JDK	21 or later
Docker	with Compose v2 (<code>docker compose</code> , not <code>docker-compose</code>)
Node	20+, only for the admin UI and for building this manual
Memory	~6 GB free; the two Aeron tmpfs volumes are sized 1 GB each

The JVM needs two module opens for Agrona's buffer intrinsics and the Aeron driver. They are already set on every Gradle test and run task, and in every start script the build produces:

```
--add-opens java.base/jdk.internal.misc=ALL-UNNAMED
--add-opens java.base/sun.nio.ch=ALL-UNNAMED
```

Without them the first `UnsafeBuffer` allocation throws `IllegalAccessError`.

2.2 Build

```
./gradlew build          # compile, generate SBE codecs, run the test suite
./gradlew installDist    # start scripts under */build/install, which the images copy
```

`./gradlew build` is npm-free by design; the Vue frontend in `web/` is a normal frontend project and is built separately.

WARNINGS ARE ERRORS

Every module compiles with warnings as errors, `control` included. Do not disable it to get a build through — a silently un-inlined function would break the engine's zero-allocation profile without failing anything.

2.3 The whole stack at once

```
./gradlew installDist
cd deploy
docker compose up -d          # roughly 40 seconds to a trading market
```

Admin UI	http://localhost:8081 — admin / most-dev-password
Control API	http://localhost:8080
Teardown	<code>docker compose down</code> — add <code>-v</code> to discard the books as well

`docker compose up` also runs a one-shot `seed` container that creates shard 0 in the control plane, imports the same security file the processes booted from, publishes a release, seeds reference prices and walks the session to `CONTINUOUS`. It does all of that over the REST API, so `deploy/init/seed.sh` doubles as a worked example of driving the exchange the supported way.

```
seed: creating shard 0
seed: importing the shard security file the processes booted from
seed: import said: {"shardId":0,"fingerprint":"3e04cf2b9902f08a","inserted":[],"updated":
["AAPL","MSFT"]}
seed: publishing release
seed: seeding definition for security 1
seed: seeding definition for security 2
seed: session → PRE_OPEN
seed: session → OPEN_AUCTION
seed: session → CONTINUOUS
seed: done -- shard 0 is CONTINUOUS with AAPL and MSFT at 100.00
```

What each container is

Service	What it is
<code>cluster-host</code>	The shard's media driver, archive and consensus module. Holds the shard0 network alias.
<code>engine</code>	The service container only. Attaches to <code>cluster-host</code> over shared memory.
<code>gateway</code>	Order entry, with its cluster identity <code>gw-0</code> .
<code>market-data</code>	L1/L2/L3 and the snapshot feed.
<code>discovery</code>	The directory broadcast.
<code>control</code> , <code>postgres</code> , <code>web</code>	The control plane, its database, and the SPA behind <code>nginx</code> .
<code>control-driver</code>	The control plane's <i>own</i> media driver, so it reaches the shard over the network the way an adapter will rather than by mounting the engine's shared memory.
<code>most</code>	A run-only container for the CLI. Started on demand by <code>deploy/most</code> .

MEDIA DRIVER

The Aeron process that owns the shared-memory buffers and the sockets. Every process on a node talks to one through a directory of memory-mapped files. It allocates, which is why it is never inside the engine binary.

2.4 Verifying the stack came up

Every process prints its geometry fingerprint at startup. They must all agree.

```
$ docker compose logs cluster-host engine gateway market-data discovery | grep -i fingerprint

cluster:      participants=/config/shard-0-participants.properties fingerprint=cb7d8bceec5e98b4
gateways=[gw-0] reload=5000ms
matching-engine: shard=0 fingerprint=3e04cf2b9902f08a securities=[AAPL, MSFT] serviceId=0
participantRegistry=cb7d8bceec5e98b4
gateway:      shard=0 fingerprint=3e04cf2b9902f08a securities=[AAPL, MSFT] in=aeron:udp?
endpoint=0.0.0.0:20001:20
market-data:  shard=0 fingerprint=3e04cf2b9902f08a securities=[AAPL, MSFT] in=aeron:ipc:12
```

Two distinct fingerprints appear here and both are correct: `3e04cf2b9902f08a` is the **shard's geometry**, and `cb7d8bceec5e98b4` is the **participant registry's**. They are deliberately separate values — rotating a gateway's secret is not a change of geometry, and must not invalidate a release.

The five processes should each print a readiness line:

```
cluster:      started, awaiting shutdown signal
matching-engine: started, awaiting shutdown signal
gateway:      started
market-data:  started
discovery:    started
```

2.5 The operator CLI

`most` is the operator command-line tool. Against the dev stack, use the `deploy/most` wrapper — it runs the CLI in a container with its own media driver, on the outside of the shard, exactly where a FIX adapter will sit.

Every subcommand begins by waiting for a **discovery broadcast**, because the routing table is the only thing that maps a symbol to the shard and gateway serving it. A `most` command that appears to hang is almost always waiting for that.

```
$ ./most securities --verbose

SYMBOL      ISIN          SHARD CCY    NAME
AAPL        US0378331005  0      USD    Apple Inc.
            id=1 tick=0.01 floor=0.00 levels=32768
            orders → aeron:udp?endpoint=shard0:20001:20 reports ←
aeron:udp?...control=shard0:20002...:21
MSFT        US5949181045  0      USD    Microsoft Corporation
            id=2 tick=0.01 floor=0.00 levels=32768
            orders → aeron:udp?endpoint=shard0:20001:20 reports ←
aeron:udp?...control=shard0:20002...:21

2 securities, universe version 9181280125937456696
```

The **universe version** is a 64-bit hash of the whole published directory. It is how a subscriber detects that routing has changed underneath it.

2.6 Driving a trade

Rest an order, then cross it from a second participant.

```
$ ./most send --symbol MSFT --side sell --price 100.50 --qty 10 --clordid 2001 --participant 7 --follow 2

sending sell 10 MSFT @ 100.50 clOrdId=2001
sent to shard 0 via aeron:udp?endpoint=shard0:20001
  NEW  MSFT  orderId=11 clOrdId=2001  leaves 10 cum 0 of 10

$ ./most send --symbol MSFT --side buy --price 100.50 --qty 4 --clordid 2002 --participant 8 --follow 2

sending buy 4 MSFT @ 100.50 clOrdId=2002
sent to shard 0 via aeron:udp?endpoint=shard0:20001
  NEW  MSFT  orderId=12 clOrdId=2002  leaves 4 cum 0 of 4
  TRADE MSFT  orderId=12 clOrdId=2002  last 4 @ 100.50 cum 4 leaves 0
```

`orderId` in the `NEW` report is the `exchangeOrderId` and is what a cancel refers to. Cancelling the partially filled maker shows both quantities the engine states on every report:

```
$ ./most cancel --symbol MSFT --side sell --order-id 11 --orig-clordid 2001 --participant 7 --follow 2

cancelling orderId=11 on MSFT
  CANCELED MSFT  orderId=11  leaves 0 cum 4 of 10
```

CUMQTY AND ORIGQTY

`cumQty` is the quantity filled so far and `origQty` the quantity originally submitted; both are **stated by the engine on every execution report** and are never derived by a client. A terminal report carries `leavesQty = 0` whether the order filled or was cancelled, so subtracting `origQty - leavesQty` would report this cancelled order as fully filled. An `origQty` of `0` means *unknown*, which reaches a client as an explicit `UNKNOWN` enrichment rather than as a number.

A rejection is an execution report like any other:

```
$ ./most send --symbol AAPL --side buy --price 500.00 --qty 1 --clordid 3001 --participant 7 --follow 2

REJECTED  AAPL  orderId=0 clOrdId=3001  reason PRICE_OUT_OF_BOUNDS
```

An unknown symbol never reaches the wire — the CLI resolves symbols against the directory first:

```
$ ./most send --symbol TSLA --side buy --price 100.00 --qty 1 --clordid 3002
most: unknown symbol 'TSLA' -- try `most securities`
```

2.7 Watching the book

`most book` rebuilds order books from the L2 depth feed and its periodic snapshot, using the same assembler any other consumer would use.

```
$ ./most book --symbol AAPL,MSFT --depth 5 --refresh 500
```

```
— AAPL (id 1) —
  qty ords      bid | ask      qty      ords
  128  2      100.64 | 101.00   40       1
   54  1      100.59 | 101.10   40       1
   50  1      100.55 | 101.20   40       1
  103  2      100.51 | 101.67  103       2
   43  1      100.50 | 101.68    1       1
spread 0.36  last 101.61 x 29  updates 41

— MSFT (id 2) —
  qty ords      bid | ask      qty      ords
                        100.50    6       1
spread n/a   last 100.50 x 4   updates 1
```

If it prints `waiting for the depth feed...` it has not yet received a snapshot for that security. The snapshot feed cycles one security per interval, so a joiner waits at most one full pass.

SYNCHRONISED

A depth subscriber is *synchronised* once it has installed a snapshot image and can apply increments to it. Before that it publishes **no depth at all** rather than a partial ladder: “waiting for a snapshot” and “no liquidity” are different answers and must never render the same way.

2.8 The admin UI

The console is at <http://localhost:8081>. It is a Vue SPA over the control plane’s REST API, served same-origin — the session and CSRF cookies are same-origin mechanisms, so the SPA and the API sit behind one host.

most
exchange control

Operator

Password

Sign in

Figure 2.1 The sign-in screen. Every route but this one requires an authenticated operator; the router guard exists so an operator sees a form rather than empty tables, and the server's 401 is what actually protects the data.

Status is the first screen and answers a question no other screen does: what the exchange is actually doing, as opposed to what the database says it should be. It is derived from the L3 book event feed.

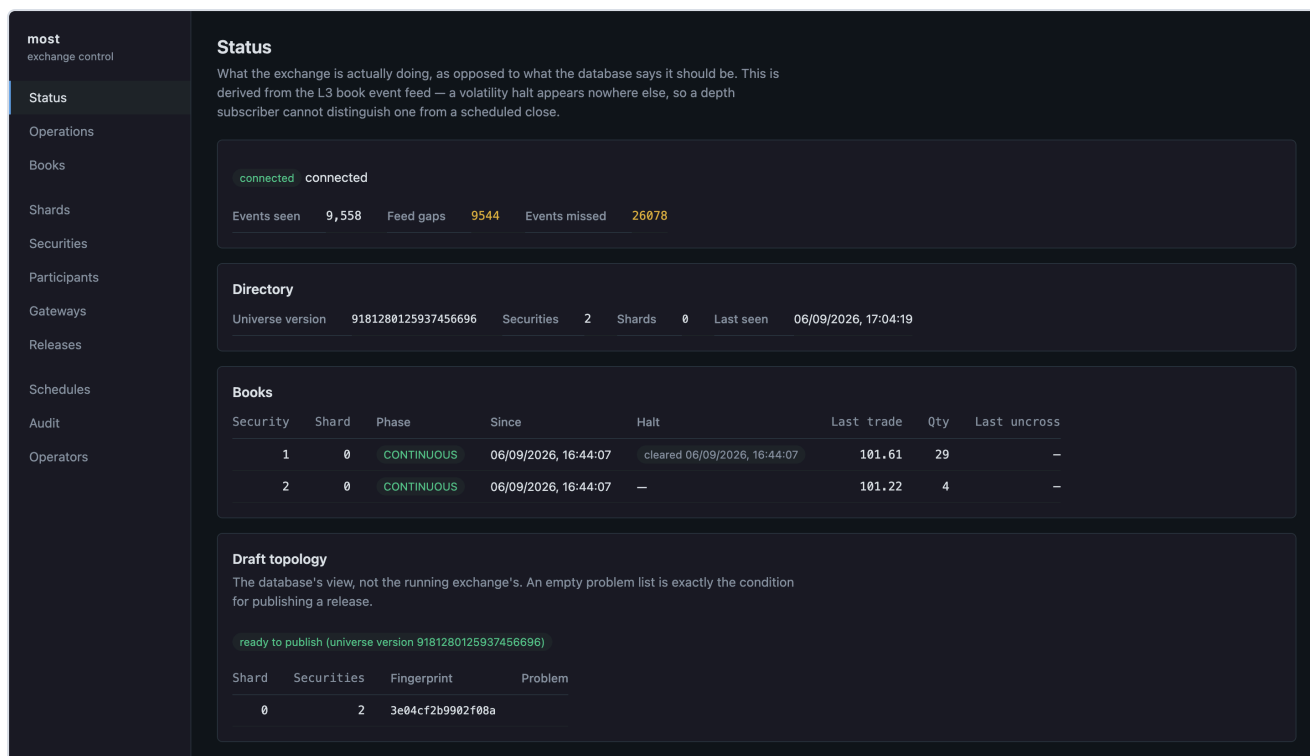


Figure 2.2 Status. Observed phase per book from the L3 feed, the directory the shard is broadcasting, and the draft topology's readiness to publish.

Books renders live depth, streamed over server-sent events, conflated to four images a second. A book that is not synchronised draws no ladder at all.

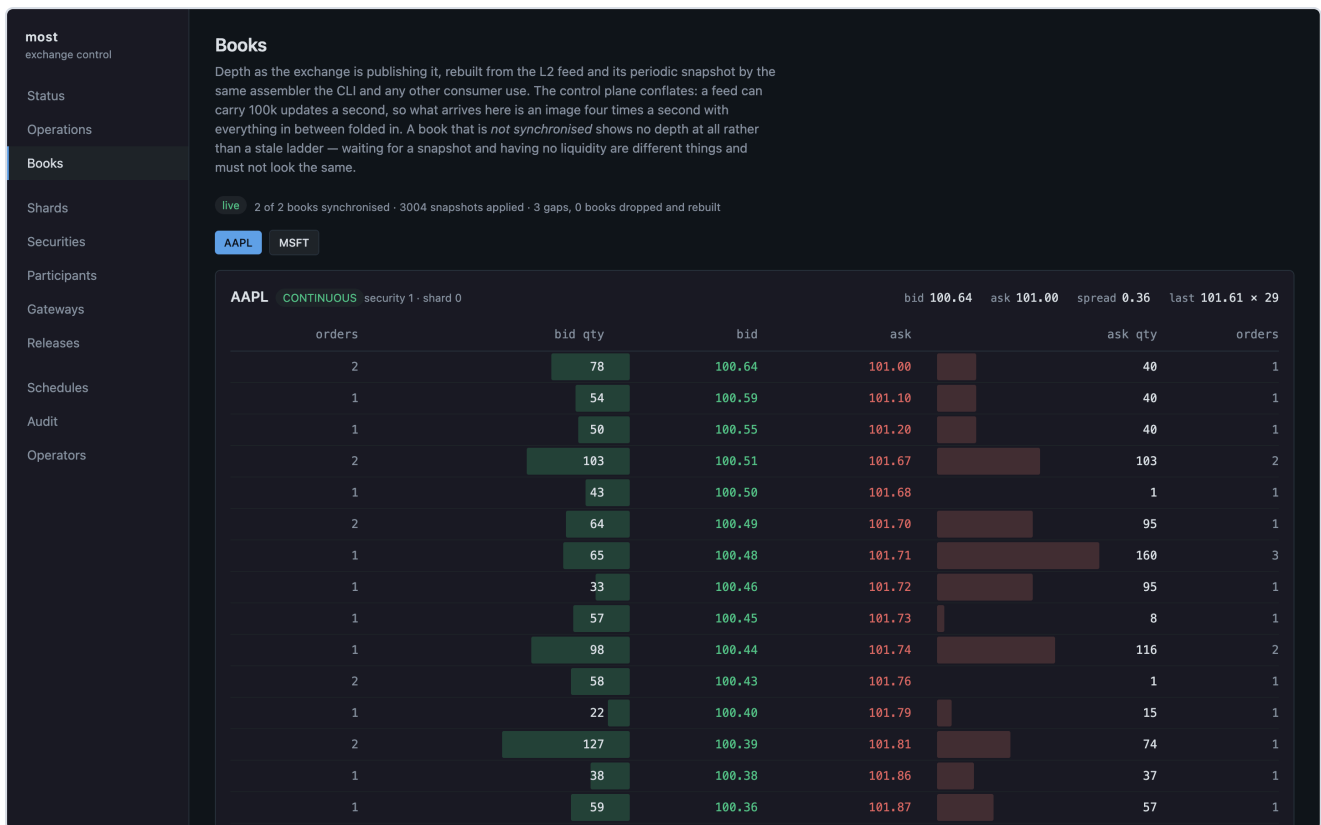


Figure 2.3 Books. Depth rebuilt by the same assembler the CLI uses, conflated on the feed thread so a browser is never on the market data critical path.

Operations is the only screen that touches a running market.

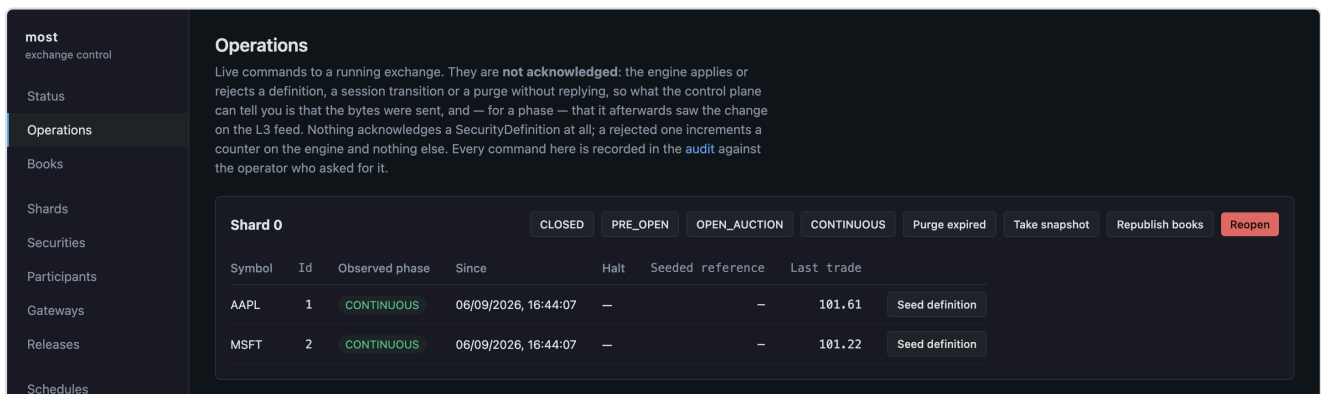


Figure 2.4 Operations. Session transitions, definition seeding, the expiry purge, a snapshot request, a book image request, and halt recovery.

The remaining screens are covered where they are used: Shards, Securities, Participants, Gateways and Releases in section 4; Schedules, Audit and Operators in section 5.

2.9 What is development-only

Development stack	Production
One cluster node	Three or five members on separate machines (3.2)
config/gateway-0.secret committed to the repository	A secret issued by the control plane, delivered out of band (4.3, 4.10)
A fixed, published admin password	CONTROL_ADMIN_PASSWORD unset, so one is generated and logged once (4.9)
CONTROL_COOKIE_SECURE=false	true, behind TLS
Dynamic MDC for market data and the gateway's outbound leg	UDP multicast with an IGMP querier (3.7)
The scheduler disabled	Enabled — it is what bounds restart time by snapshotting at each close (5.6)
Hand-written shard-0-securities.properties	A published release, pointed at by a current symlink (4.2)
ThreadingMode.SHARED, no core isolation	Dedicated threading and isolated cores (3.4)

ONE MEDIA DRIVER IS ONE NETWORK IDENTITY

In the development stack the five shard processes share a media driver that runs in the `cluster-host` container, so **every UDP endpoint the shard exposes is bound there** and must name `shard0`, never the individual process's container name. A channel naming `gateway:20002` fails with `Cannot assign requested address`. This is not a Docker workaround; it is what “these five processes are one node sharing `/dev/shm`” means once the processes are in separate containers.

2.10 Running without Docker

Useful when iterating on one process. Five terminals, started in this order — the engine attaches to a consensus module that must already exist.

```
# 1. cluster host: media driver + archive + consensus module
./tools/build/install/tools/bin/most cluster --dir build/cluster-host \
  --participants config/shard-0-participants.properties
# wait for: cluster: started, awaiting shutdown signal

# 2. engine — the cluster service container only
./engine/build/install/engine/bin/engine config/engine.properties
# wait for: matching-engine: started, awaiting shutdown signal

# 3. gateway
./gateway/build/install/gateway/bin/gateway config/gateway.properties

# 4. market data
./market-data/build/install/market-data/bin/market-data config/market-data.properties

# 5. discovery
./discovery/build/install/discovery/bin/discovery config/discovery.properties
```

The control plane needs Postgres and one environment variable:

```
docker run -d --name most-control-db -p 5432:5432 \
  -e POSTGRES_DB=most_control -e POSTGRES_USER=most -e POSTGRES_PASSWORD=most postgres:16-alpine

./gradlew :control:installDist
CONTROL_ADMIN_PASSWORD=... ./control/build/install/control/bin/control
```

And the frontend, which proxies `/api` to the control plane so both are one origin:

```
cd web && npm install && npm run dev      # http://localhost:5173
```

2.11 Automated end-to-end checks

Three scripts exercise things unit tests cannot. Run the first two after changing anything that touches the wire or process lifecycle.

Script	What it proves
<code>e2e/run-e2e.sh</code>	Every process against a real single-node cluster: a trade driven through the CLI, then a short most load to confirm the harness still correlates reports to orders. Takes each binary path from an environment variable (ENGINE, GATEWAY, MARKETDATA, DISCOVERY) so the same run can drive native images.
<code>e2e/run-restart.sh</code>	That state survives a restart. Rests orders, snapshots, stops the whole node, and restarts it on the same security file; on one with a security removed while it holds orders (must refuse); on one with an emptied security removed (must start); and on one with a security added. Also places an order through one gateway and cancels it through another.
<code>e2e/run-attribution.sh</code>	Splits a client round trip into gateway, engine and everything else, writing <code>.hgrm</code> histograms that are meant to be diffed across changes.
<code>e2e/run-epsilon-soak.sh</code>	Runs the Epsilon-GC binary against a real cluster and measures the allocation slope across two runs.

`run-e2e.sh` wipes everything and starts fresh, so it says nothing about whether state survives — that is `run-restart.sh` and only `run-restart.sh`.

2.12 Shutting down

```
cd deploy
docker compose down      # keeps the books: cluster-data is a durable named volume
docker compose down -v   # discards the archive, the cluster directory and the books
```

A NODE CANNOT RESTART IMMEDIATELY AFTER THE PREVIOUS ONE STOPPED

Aeron's archive and cluster **mark files** carry a liveness timestamp, and a new process refuses to start until it ages out — roughly ten seconds, even after a clean shutdown. An automatic restart policy must wait longer than that or it will fail on `active mark file detected` and back off.

3. Production deployment

3.1 The rule that shapes everything

A shard is not orchestrated. The three cluster nodes are long-lived machines with stable hostnames, isolated cores, huge pages and a dedicated NVMe device for the archive. They are not pods, they are not autoscaled, and they are not rescheduled. Everything with no realtime requirement — the control plane, its database, the admin UI, protocol adapters — goes to an ordinary orchestrated tier.

That line is not a preference. The engine is a single thread whose working set is a cache-line packed order pool; a rescheduled workload, a shared core or a cross-socket memory access costs more than everything the matching logic does.

NOTHING IN THIS SECTION HAS BEEN RUN END TO END

The port map, member string and unit files below are derived from the code and from the development stack, not from a working production deployment. 3.10 lists the specific code changes that are blocking, and every one of them is small.

3.2 Topology

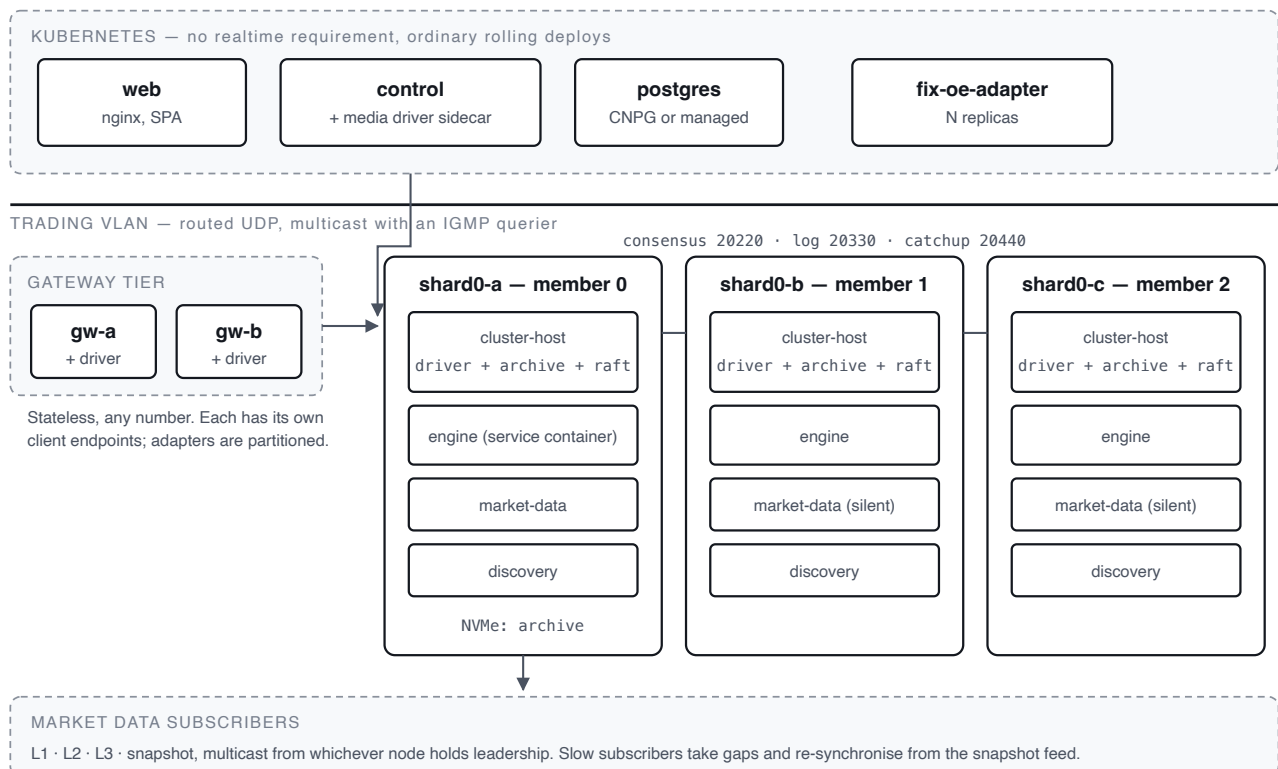


Figure 3.1 The production topology: three cluster nodes on bare metal, a separate stateless gateway tier, and everything without a realtime requirement in Kubernetes.

What runs where

Every node runs **cluster-host** and **engine**. That is what a Raft group is.

Every node runs `market-data`, **and only one of them emits**. This falls out of the design rather than needing coordination: the book event stream is `aeron:ipc` and is leader-only, muted explicitly on role change because it is a plain publication and not cluster egress. A follower's market-data process is subscribed to a silent stream and publishes nothing. On failover the new leader's engine starts publishing and its co-located market-data picks up. There is no standby logic and no cross-machine hop for a stream that was deliberately made shared-memory.

Every node may run `discovery`. It is a stateless repeating broadcast and a duplicate cycle is harmless — each cycle is staged and replaces the routing table wholesale, so a truncated cycle never replaces a good table.

`gateway` **is not on these machines and is not active/standby**. A gateway holds no per-order state, so co-locating one with a cluster node buys nothing: an `AeronCluster` client lists every member and follows the leader whether it sits beside it or a rack away. It is its own tier, sized and restarted independently of the Raft group.

GATEWAYS MUST HAVE DISJOINT CLIENT ENDPOINTS

`gateway.client.inbound.channel` is a single address. Two gateways subscribed to **one** inbound channel each receive every order and forward both — that is duplicate orders, not redundancy. Give each gateway its own client endpoints and partition adapters across them. Losing one costs its adapters a reconnect and nothing else; a peer reports correctly on orders it never saw.

⚠ THE DIRECTORY ADVERTISES ONE GATEWAY PER SHARD

`ShardEntry` carries a single order-entry channel and `DirectoryClient` keeps one route per shard, so several gateways for one shard cannot all be advertised. A virtual address in front of the gateway tier is the interim answer; naming them individually is a wire change.

3.3 Machine preparation

Hardware

	Recommendation	Why
CPU	16+ physical cores, x86-64-v3 or better, one socket	~12 threads need cores; one socket avoids NUMA on the hot path
RAM	64 GB minimum	~0.9 GB per shard of pools, plus Aeron log buffers, plus page cache for the archive
Storage	NVMe SSD dedicated to the archive	The archive writes every replicated message and is the single largest contributor to round-trip latency
Network	Dual 10 GbE, one dedicated to the trading VLAN	Multicast market data and consensus must not share a link with management traffic

Single socket matters more than it looks. A cross-socket memory access on the matching path costs more than the matching itself.

Kernel and boot

```
# /etc/default/grub - GRUB_CMDLINE_LINUX
isolcpus=2-13 nohz_full=2-13 rcu_nocbs=2-13
hugepagesz=1G hugepages=8 default_hugepagesz=1G
intel_idle.max_cstate=0 processor.max_cstate=1 idle=poll
mce=ignore_ce
```

HUGE PAGES ARE MANDATORY, NOT OPTIONAL

A 0.9 GB order pool on 4 KB pages is roughly 230,000 TLB entries, for a structure that was cache-line packed specifically to cost one cache miss per order. The TLB miss undoes the entire design.

`idle=poll` and the C-state pinning trade power for the absence of wake-up latency. On a machine whose only job is one exchange that is the right trade — and it is also why these machines should run nothing else.

```
tuned-adm profile latency-performance

# NIC: no interrupt coalescing on the trading interface, IRQs pinned to housekeeping cores
ethtool -C ens1f0 rx-usecs 0 rx-frames 1
ethtool -G ens1f0 rx 4096 tx 4096
```

3.4 Core allocation

Cores 0–1 are housekeeping — kernel, IRQs, ssh, metrics agents. Cores 2–13 are isolated.

Cores	Thread
2, 3, 4	Media driver conductor, sender, receiver (<code>ThreadingMode.DEDICATED</code>)
5, 6	Archive conductor, recorder (<code>ArchiveThreadingMode.DEDICATED</code>)
7	Consensus module
8	Engine service container — the matching thread
9, 10	Gateway, and its Aeron client conductor
11, 12	Market-data, and its client conductor
13	Discovery

Core 8 is the one that matters. Everything else is plumbing around it.

IDLE STRATEGIES ARE NOT PORTABLE BETWEEN MACHINES

With dedicated cores, `BusySpinIdleStrategy` is correct for the driver threads, the consensus module and the service container. **Without** isolated cores it is actively harmful — spinning threads starve the kernel's own softirq processing. Never copy production idle strategies onto a development machine.

⚠ THREADING MODES AND IDLE STRATEGIES ARE HARDCODED

`most cluster` sets `ThreadingMode.SHARED` and `ArchiveThreadingMode.SHARED` with no override, and idle strategies are constants rather than configuration. Both are correct for a laptop and wrong for the table above.

3.5 Aeron buffer sizing

Aeron's defaults are sized for throughput, not for a fixed memory budget. On 64 GB the UDP defaults are fine; the IPC default deserves attention, because a 64 MB IPC term means roughly a 192 MB log buffer for the book event stream alone.

```
aeron.term.buffer.length=16m
aeron.ipc.term.buffer.length=16m
aeron.socket.so_sndbuf=2m
aeron.socket.so_rcvbuf=2m
aeron.rcv.initial.window.length=2m
```

A DRIVER THAT CANNOT MAP A BUFFER DOES NOT FAIL POLITELY

It takes SIGBUS on the next page touch and reports `InternalError: a fault occurred in an unsafe memory access operation`, which reads like a JVM bug rather than a sizing problem. Size the shared-memory mount for the number of subscriptions the process will hold: every subscription image is a log buffer of three terms.

3.6 Cluster configuration

Member string

Aeron's format is `memberId,ingress,consensus,log,catchup,archiveControl`, members separated by `|`. The ports are the CLI's existing defaults and are fine to keep — the three members are on different machines, so they do not need to differ.

```
0,shard0-a:20110,shard0-a:20220,shard0-a:20330,shard0-a:20440,shard0-a:8010|\
1,shard0-b:20110,shard0-b:20220,shard0-b:20330,shard0-b:20440,shard0-b:8010|\
2,shard0-c:20110,shard0-c:20220,shard0-c:20330,shard0-c:20440,shard0-c:8010
```

Pass it identically to all three with `most cluster --members`, varying only the member id.

⚠ 'MOST CLUSTER' CANNOT BE A MEMBER OTHER THAN 0

The cluster host hardcodes member id 0 and exposes no `--member-id`, so all three nodes would claim to be member 0. This is blocking for a multi-node deployment.

Port map

Every one of these is bound by the **media driver** on its own machine.

Port	Bound by	Purpose
20110	consensus	Cluster ingress — gateways and the control plane connect here
20220	consensus	Consensus, member to member
20330	consensus	Log replication
20440	consensus	Catchup
8010	archive	Archive control
20001	gateway	Client inbound — adapters publish orders here
20002	gateway	Client outbound control address — execution reports
40000	discovery	Directory broadcast
40001 / 40002 / 40003	market-data	L1 / L2 / L3
40004	market-data	L2 recovery snapshot feed

20110, 20001 and 20002 are the only ports anything outside the trading VLAN needs to reach. Firewall the rest to the three machines and the market-data subscriber group.

Gateway ingress

The gateway lists every member so its cluster client can find the leader:

```
gateway.ingressChannel=aeron:udp
gateway.ingressEndpoints=0=shard0-a:20110,1=shard0-b:20110,2=shard0-c:20110
```

3.7 Market data: multicast

The development stack uses dynamic MDC because a Docker bridge does not route multicast. On a real VLAN, use what the design specifies:

```
md.l1.channel=aeron:udp?endpoint=239.10.0.1:40001|interface=10.20.0.0/24
md.l1.streamId=1
md.l2.channel=aeron:udp?endpoint=239.10.0.2:40002|interface=10.20.0.0/24
md.l2.streamId=2
md.l3.channel=aeron:udp?endpoint=239.10.0.3:40003|interface=10.20.0.0/24
md.l3.streamId=3
md.snapshot.channel=aeron:udp?endpoint=239.10.0.4:40004|interface=10.20.0.0/24
md.snapshot.streamId=4
md.snapshot.cycleMs=1000
```

The `interface=` qualifier is not optional on a multi-homed machine — without it Aeron picks an interface and it may not be the trading VLAN.

CONFIGURE IGMP SNOOPING AND A QUERIER ON THE VLAN

Without an IGMP querier, snooping switches age out group membership and the feed silently stops reaching subscribers, usually minutes after everything looked fine.

DO NOT CHANGE AERON'S FLOW CONTROL

It defaults to `MaxMulticastFlowControl` — the fastest receiver governs — and that is correct here.

`MinMulticastFlowControl` would let the slowest subscriber throttle the publisher, reintroducing exactly the coupling that moving market data out of the engine was meant to remove. The accepted trade-off is that slow subscribers take unrecoverable gaps; gap detection and snapshot re-synchronisation are subscriber responsibilities, and `reference` provides both (`FeedSequenceTracker` , `DepthFeedAssembler`).

⚠ A FEED IS ONE CHANNEL

`market-data` cannot publish the same feed as both multicast and dynamic MDC, so a subscriber that cannot receive multicast cannot be served alongside one that can.

3.8 Process management

One `systemd` unit per process. The engine's is the template; the rest differ only in `ExecStart` and `CPUAffinity`.

```
# /etc/systemd/system/most-engine.service
[Unit]
Description=most-exchange matching engine (shard 0)
After=most-cluster-host.service
Requires=most-cluster-host.service

[Service]
Type=simple
User=most
ExecStart=/opt/most/bin/engine /etc/most/engine.properties
CPUAffinity=8
LimitMEMLOCK=infinity
LimitNOFILE=1048576

KillSignal=SIGTERM
TimeoutStopSec=60
Restart=on-failure
RestartSec=15s

[Install]
WantedBy=multi-user.target
```

`RestartSec=15s` is not arbitrary: it is longer than the roughly ten-second mark-file liveness timeout, so an automatic restart does not fail on `active mark file detected` and then back off.

THE WHOLE SHUTDOWN PATH HANGS OFF AN ORDERLY SIGTERM

Snapshot counters, latency summaries and the service container's own close all run inside a shutdown barrier that SIGTERM releases. `SIGKILL` skips them and leaves the mark file carrying a live timestamp, so the next start refuses for ten seconds. For native images `--install-exit-handlers` is load-bearing and is set in the root build; without it SIGTERM kills the image outright.

Configuration management

Two kinds of file that must not be confused:

- **Released artifacts** — `shard-N-securities.properties`, `shard-N-participants.properties`, the discovery registry. Published by the control plane as immutable numbered releases. Your configuration-management tool *fetches* a release and points `current` at it; it never authors one.
- **Node-local config** — `engine.properties`, `gateway.properties`, Aeron and cluster directories, feed channels. Lives in the repository and is templated per host.

3.9 Bringing a shard up

Order matters, and most of it is the mark-file timeout in disguise.

```
# 1. All three cluster hosts, within a few seconds of each other. A member that starts minutes
#   late will catch up, but the group will not form until two are present.
systemctl start most-cluster-host          # on shard0-a, -b, -c

# 2. Wait for a leader. Not optional: the engine's service container attaches to a consensus
#   module that must already exist.
journalctl -u most-cluster-host -f | grep -m1 'Leader'

# 3. Engine service containers, all three.
systemctl start most-engine

# 4. The rest, on their assigned nodes.
systemctl start most-market-data           # all three; followers stay silent
systemctl start most-discovery             # all three is safe

# 5. Gateways, on their own machines. Order does not matter and neither does how many.
systemctl start most-gateway

# 6. Control plane last – it is a client of everything above.
kubectl -n most rollout status deploy/control
```

Verify geometry before trusting anything

```
ansible shard0 -a "journalctl -u most-engine -n 200 --no-pager" | grep -i fingerprint | sort -u
```

One distinct value, or stop and fix it. Raft will not catch this.

Establish the session baseline

A cold cluster has emitted no `SessionChanged`, so its phase is genuinely unknown to anything watching the feed. The scheduler **waits** rather than assuming `CLOSED`, because a control-plane restart mid-session looks identical from the outside and assuming would shut a live market.

Issue one session command deliberately as the last step of bringing a cold shard up. After that, the feed keeps the control plane current and the calendar takes over.

3.10 Blocking gaps

An honest list. None of these are large; all of them block a production deployment.

#	Gap	Blocks
1	most cluster hardcodes member id 0 and exposes no <code>--member-id</code>	3.6 — a multi-node cluster
2	Driver and archive threading modes are hardcoded <code>SHARED</code>	3.4 — dedicated threads
3	Idle strategies are constants, not configuration	3.4 — busy-spin only where cores are isolated
4	engine.march is x86-only and passed straight to native-image	A second architecture
5	The directory advertises one gateway per shard	3.2 — naming several gateways
6	A feed is one channel; multicast and MDC cannot coexist	3.7 — mixed subscribers

#	Gap	Blocks
7	No health endpoint on the core processes	6.8 — mechanised alerting; today everything is log lines and shutdown counters
8	None of this has been run as described	All of section 3

4. Configuration in full

This section is a reference. Every key each process reads, every artifact a release contains, and every environment variable the control plane honours.

4.1 How a process is configured

Each of the four core processes takes **one positional argument**: the path to a properties file.

```
engine      /etc/most/engine.properties
gateway     /etc/most/gateway.properties
market-data /etc/most/market-data.properties
discovery   /etc/most/discovery.properties
```

After the file is read, **JVM system properties in the same namespace override individual keys**. The prefix is the process's own: `engine.`, `gateway.`, `md.`, `discovery.`. This is how a deployment overrides one value without templating a whole file:

```
engine -Dengine.metrics=true /etc/most/engine.properties
```

The cluster host is different: it is a subcommand of the operator CLI and takes flags, not a file. The control plane is a Spring Boot application and takes environment variables.

A MISSING REQUIRED KEY STOPS THE PROCESS, AND THAT IS THE DESIGN

`engine.securitiesFile` and its equivalents have no default. Nothing in the shard security file has a default either — a wrong tick size misprices every order in silence, and a missing ISIN would be published to every downstream adapter.

4.2 The shard security file

The single most important artifact. It is read at boot by the engine, the gateway, market-data and discovery **alike**, so a shard has one definition and one fingerprint rather than four lists kept in step by hand. In production it is published by the control plane; the path each process is given should be a `current` symlink into a release directory.

```
shard.id=0
shard.securities=1,2

security.1.symbol=AAPL
security.1.isin=US0378331005
security.1.name=Apple Inc.
security.1.currency=USD
security.1.priceFloor=0
security.1.tickSize=1000000
security.1.levelCount=32768
security.1.maxOrders=1000000
```

Key	Type	Constraint
shard.id	int	Non-negative
shard.securities	int list	1 to 10 ids, comma-separated, no duplicates
security.N.symbol	string	Non-blank, at most 16 characters — the wire field's length
security.N.isin	string	12 characters, check-digit validated at boot
security.N.name	string	At most 48 characters
security.N.currency	string	Exactly 3 characters
security.N.priceFloor	int64	The price of ladder level 0, fixed point, 8 decimals
security.N.tickSize	int64	Positive. 1000000 is 0.01
security.N.levelCount	int	Positive. Ladder levels allocated per side
security.N.maxOrders	int	Positive. Order pool high-water mark; up to 1,000,000

LADDER GEOMETRY

`priceFloor`, `tickSize` and `levelCount` define a flat array of price levels per side, indexed by ticks above the floor. The ladder's range is `priceFloor` to `priceFloor + tickSize × levelCount`. It must be configured **strictly wider than the static circuit-breaker band**, so band rejection always fires before the ladder can overflow.

GEOMETRY CHANGES ARE ENFORCED AGAINST THE SNAPSHOT

On restart the engine reconciles this file against the snapshot it is restoring and **refuses to start** rather than lose state: a security removed while it holds resting orders, a changed `priceFloor`, `tickSize`, `levelCount` or `maxOrders`, an order that falls outside the new ladder, or resting-order counts that do not add up. A refusal is the system working. The two legitimate cases are quiet — an *empty* book leaving the shard, and a new security joining — and each is logged.

Reference prices and collar widths are deliberately **not** in this file. They change during a session and every node must apply them at the same log position, so they arrive as `SecurityDefinition` commands through the replicated log (5.3).

4.3 The participant registry

Which gateway may connect as which identity, and which participants it speaks for. Read by **two** processes for two different reasons: the consensus module authenticates a connecting gateway against it, and the engine resolves the resulting principal into a participant list at session open.

```
shard.id=0
registry.gateways=gw-0

gateway.gw-0.secret=3b2c637a03bcb9c167c9bd5f84de22f6bbdc18169d36526cafc164cd68725367
gateway.gw-0.participants=7,8
```

Key	Constraint
shard.id	Must match the shard of the process reading it, or the process refuses to start
registry.gateways	At least one id, comma-separated
gateway.<id>.secret	The SHA-256 hex digest of the shared secret, 64 lowercase hex characters

Key	Constraint
gateway.<id>.participants	At least one positive participant id. A participant belongs to at most one gateway

Gateway ids are at most 64 characters from a restricted alphabet. The file holds the digest, never the secret; the secret itself is delivered only to the gateway that presents it.

PARTICIPANT BINDING

A gateway presents `gatewayId:secret` as its cluster credentials. The consensus module verifies them and stamps the gateway id on the session as its **encoded principal**. Aeron carries that principal in the session-open event through the replicated log, so every node derives the identical participant-to-session map without any snapshot state of its own. This is what makes a participant's fills deliverable when it has said nothing since the gateway last connected.

WRONG CREDENTIALS ARE REJECTED, NEVER DOWNGRADED

A gateway with a bad secret fails to connect. It is never quietly downgraded to an anonymous session — an anonymous gateway trades perfectly well and loses only the fills of whichever participants have gone quiet, which is invisible until someone reconciles a `cumQty`. **No** credentials at all still authenticates anonymously, because the control plane and the CLI connect to send operator commands and are addressed by nobody.

Reloading while a node runs

The consensus module and the engine both **re-read this file while running**. They compare content by fingerprint rather than mtime — a release is published to a new directory and put in force by moving a symlink, so mtime says nothing — and swap an immutable registry behind a volatile reference.

A file that cannot be parsed, or one for another shard, is **reported and ignored**; the registry in force keeps authenticating. Standing down on a bad file would turn a typo into a shard that authenticates nobody. Watch `registryReloadFailures` rather than assuming a silent success.

The gateway does **not** reload; it restarts, which costs nothing because it holds no state.

4.4 Engine

```
engine.securitiesFile=/etc/most/current/shard-0-securities.properties
engine.participantRegistry=/etc/most/current/shard-0-participants.properties
engine.aeronDir=/dev/shm/aeron-most
engine.clusterDir=/var/lib/most/cluster
engine.bookEvent.channel=aeron:ipc
engine.bookEvent.streamId=12
```

Key	Default	Meaning
engine.securitiesFile	<i>required</i>	The shard security file (4.2)
engine.participantRegistry	none	The participant registry (4.3). Omit and routes are learned from traffic alone
engine.participantRegistry.reloadMs	5000	Registry poll interval; 0 disables reloading
engine.aeronDir	driver default	Must be the media driver this node's processes share

Key	Default	Meaning
<code>engine.clusterDir</code>	<code>cluster</code>	The consensus module's cluster directory, written by the cluster host
<code>engine.serviceId</code>	<code>0</code>	The clustered service id
<code>engine.bookEvent.channel</code>	<code>aeron:ipc</code>	Where book events are published. Keep it IPC: market-data is on this node
<code>engine.bookEvent.streamId</code>	<code>12</code>	
<code>engine.auction.maxPasses</code>	<code>64</code>	Safety valve on the uncross fixed-point loop, not part of the algorithm
<code>engine.backpressure.alertThreshold</code>	<code>1000000</code>	Consecutive back-pressured publications before alerting
<code>engine.metrics</code>	<code>false</code>	Hot-path timing; two clock reads per message
<code>engine.metrics.stages</code>	<code>false</code>	Adds the admit/match/settle partition of a new order; two more clock reads
<code>engine.metrics.file</code>	<code>none</code>	Where percentile distributions are written at shutdown

METRICS ARE NODE-LOCAL AND DELIBERATELY OUTSIDE THE FINGERPRINT

The engine reads `System.nanoTime()` to record these histograms, which the determinism rules otherwise prohibit. The rule is that time must not influence **replicated state**, not that it may not be observed: the histograms are write-only — never read by a branch, never snapshotted, never on a feed. Enabling metrics on one node and not another must be incapable of changing the log, the books or a snapshot. That is why `engine.metrics*` is excluded from the fingerprint, and it is the test any probe added later must pass.

4.5 Gateway

```
gateway.securitiesFile=/etc/most/current/shard-0-securities.properties
gateway.participantRegistry=/etc/most/current/shard-0-participants.properties
gateway.gatewayId=gw-a
gateway.credentialTokenFile=/etc/most/gw-a.secret

gateway.aeronDir=/dev/shm/aeron-gw-a
gateway.ingressChannel=aeron:udp
gateway.ingressEndpoints=0=shard0-a:20110,1=shard0-b:20110,2=shard0-c:20110
gateway.egressChannel=aeron:udp?endpoint=0.0.0.0:0

gateway.client.inbound.channel=aeron:udp?endpoint=0.0.0.0:20001
gateway.client.inbound.streamId=20
gateway.client.outbound.channel=aeron:udp?endpoint=239.10.0.5:20002
gateway.client.outbound.streamId=21
```

Key	Default	Meaning
<code>gateway.securitiesFile</code>	<i>required</i>	The shard security file. Orders for anything else are rejected
<code>gateway.participantRegistry</code>	<code>none</code>	Checked at boot so a typo in the id stops the process here
<code>gateway.gatewayId</code>	<code>none</code>	This gateway's registry id. Unset means an anonymous connection

Key	Default	Meaning
gateway.credentialToken	none	The shared secret as a literal. Prefer the file form
gateway.credentialTokenFile	none	The shared secret from a file; trailing whitespace trimmed
gateway.aeronDir	driver default	This gateway's own media driver
gateway.ingressChannel	aeron:udp	Cluster ingress
gateway.ingressEndpoints	none	Every member's ingress endpoint, so the client can find the leader
gateway.egressChannel	aeron:udp? endpoint=localhost:9020	Cluster egress
gateway.client.inbound.channel	aeron:ipc	Where adapters publish orders . Must be unique per gateway
gateway.client.inbound.streamId	20	
gateway.client.outbound.channel	aeron:ipc	Where the gateway publishes execution reports
gateway.client.outbound.streamId	21	
gateway.metrics	false	Hot-path timing on both legs
gateway.metrics.file	none	Percentile distributions at shutdown

`gateway.gatewayId` and one of the two credential keys must be set **together**: an id with no secret cannot authenticate, and a secret with no id has nothing to authenticate as. The process refuses to start otherwise, as it does when the id is absent from the registry or the registry is for another shard.

THE GATEWAY HAS NO STORAGE CONFIGURATION

It holds no per-order state. `origQty` and `cumQty` are stated by the engine on every execution report, so a gateway can be restarted or replaced at will and several can serve one shard. If you are migrating a configuration that sets `gateway.journalFile` or `gateway.journalSlots`, delete those keys; they no longer exist.

4.6 Market data

```
md.securitiesFile=/etc/most/current/shard-0-securities.properties
md.aeronDir=/dev/shm/aeron-most
md.bookEvent.channel=aeron:ipc
md.bookEvent.streamId=12
```

Key	Default	Meaning
md.securitiesFile	<i>required</i>	Depth geometry comes from here, so ladder level N in the engine is depth level N here by construction
md.aeronDir	driver default	The node's shared media driver
md.bookEvent.channel / .streamId	aeron:ipc / 12	In from the engine

Key	Default	Meaning
md.l1.channel / .streamId	aeron:udp? endpoint=239.10.1.1:40001 / 1	Top of book and last trade
md.l2.channel / .streamId	aeron:udp? endpoint=239.10.1.2:40002 / 2	Aggregated depth increments
md.l3.channel / .streamId	aeron:udp? endpoint=239.10.1.3:40003 / 3	Book events forwarded verbatim
md.snapshot.channel / .streamId	aeron:udp? endpoint=239.10.1.4:40004 / 4	The L2 recovery feed
md.snapshot.cycleMs	1000	One security's image per cycle, so a full pass is cycleMs × securities

L1, L2, L3

L1 is top of book plus last trade. **L2** is aggregated depth per price level, published as increments carrying absolute per-level quantities. **L3** is the engine's raw book event stream forwarded byte for byte — per-order detail, and the **only** feed on which a volatility halt is visible. A depth subscriber cannot distinguish a halt from a scheduled close, which is why the control plane subscribes to L3.

4.7 Discovery

```
discovery.shards=0
discovery.shard.0.securitiesFile=/etc/most/current/shard-0-securities.properties
discovery.shard.0.orderEntryChannel=aeron:udp?endpoint=gw-a:20001
discovery.shard.0.orderEntryStreamId=20
discovery.shard.0.executionReportChannel=aeron:udp?endpoint=239.10.0.5:20002
discovery.shard.0.executionReportStreamId=21

discovery.aeronDir=/dev/shm/aeron-most
discovery.channel=aeron:udp?endpoint=239.10.0.1:40000
discovery.streamId=100
discovery.intervalMs=2000
```

Key	Default	Meaning
discovery.shards	<i>required</i>	Shard ids to publish
discovery.shard.N.securitiesFile	<i>required</i>	The shard's own security file, so the directory and the running engine cannot describe different geometry
discovery.shard.N.orderEntryChannel / StreamId	<i>required</i>	The gateway's client inbound endpoint
discovery.shard.N.executionReportChannel / StreamId	<i>required</i>	The gateway's client outbound endpoint, in subscriber form
discovery.aeronDir	driver default	
discovery.channel / .streamId	aeron:udp? endpoint=239.10.0.1:40000 / 100	The broadcast

Key	Default	Meaning
discovery.intervalMs	5000	Broadcast cycle

THE DIRECTORY PUBLISHES GATEWAY ENDPOINTS, NEVER CLUSTER INGRESS

An adapter that connected to the cluster directly would bypass the gateway's `securityId` validation and the participant binding that decides where a maker's fills go. Channels are limited to 128 characters, which is the wire field's length.

Discovery enforces **one shard per security** across the whole universe and refuses to start otherwise.

4.8 The cluster host

`most cluster` takes flags rather than a properties file.

Flag	Default	Meaning
--dir DIR	build/cluster-host	Base directory; the three below default beneath it
--aeron-dir DIR	<dir>/driver	Media driver directory
--archive-dir DIR	<dir>/archive	Archive directory — durable
--cluster-dir DIR	<dir>/cluster	Consensus module directory — durable
--host HOST	localhost	Host name used to build default endpoints
--members STRING	single-node default	Aeron member string (3.6)
--participants FILE	none	The participant registry to authenticate gateways against
--participants-reload-ms N	5000	Registry poll interval; 0 disables
--fresh	off	Delete the archive and cluster directories on start
--keep	on	Persist them. Contradicts --fresh

THE ARCHIVE AND CLUSTER DIRECTORIES ARE THE SHARD'S ONLY RESUMPTION POINT

They persist by default and `--fresh` is how you ask to lose them. The **Aeron directory is different** and is always recreated: it is memory-mapped IPC buffers and a `cnc.dat`, and keeping it makes a restart fail with `Active media driver detected` until the previous driver's liveness timeout expires.

4.9 The control plane

Configured entirely by environment variables.

Database and web

Variable	Default	
CONTROL_DB_URL	jdbc:postgresql://localhost:5432/most_control	Flyway migrates on start
CONTROL_DB_USER / CONTROL_DB_PASSWORD	most / most	
CONTROL_PORT	8080	

Variable	Default	
CONTROL_RELEASE_DIR	./build/releases	Where numbered release directories are written
CONTROL_COOKIE_SECURE	false	Set true behind TLS

Authentication

Variable	Default	
CONTROL_ADMIN_USER	admin	The first operator, seeded once into an empty table
CONTROL_ADMIN_PASSWORD	unset	With no password set, one is generated and logged at WARN on first boot

LEAVE `CONTROL\_ADMIN\_PASSWORD` UNSET OUTSIDE DEVELOPMENT

A known default password on something that can open a market would be worse than no authentication at all, because it would look protected. Passwords are stored as Bcrypt hashes in `control_user`.

The live link to a running exchange

Variable	Default	
CONTROL_AERON_ENABLED	true	false for a database-only deployment
CONTROL_AERON_DIR	driver default	Must be a media driver this host can reach
CONTROL_DISCOVERY_CHANNEL / _STREAM	aeron:udp? endpoint=239.10.0.1:40000 / 100	The routing table
CONTROL_L3_CHANNEL / _STREAM	aeron:udp? endpoint=239.10.1.3:40003 / 3	Phase, trades and halts
CONTROL_L2_CHANNEL / _STREAM	aeron:udp? endpoint=239.10.1.2:40002 / 2	Depth increments for the console
CONTROL_SNAPSHOT_CHANNEL / _STREAM	aeron:udp? endpoint=239.10.1.4:40004 / 4	Depth images
CONTROL_L1_CHANNEL / _STREAM	aeron:udp? endpoint=239.10.1.1:40001 / 1	Last trade
CONTROL_DEPTH_PUBLISH_MS	250	How often a conflated book image is pushed to browsers
CONTROL_DEPTH_MAX_LEVELS	25	Depth per side in that image

The link is **optional**. Authoring reference data must work with no media driver anywhere near it, so a missing driver is reported by `GET /api/status` and nothing else breaks.

L2, the snapshot feed and L1 go together: increments cannot be applied without an image, an image goes stale without the increments, and a synchronised subscriber ignores snapshots — so without L1 a book would trade on screen while its last price stayed empty.

Cluster ingress, for snapshots

A snapshot request is a **cluster admin request**, not a message for the log, so it is the one thing the control plane does not send through a gateway.

Variable	Default
CONTROL_CLUSTER_INGRESS_CHANNEL	aeron:udp
CONTROL_CLUSTER_EGRESS_CHANNEL	aeron:udp? endpoint=0.0.0.0:0
CONTROL_CLUSTER_INGRESS_<shardId>	none Member endpoints, e.g. 0=shard0-a:20110,1=shard0-b:20110

A shard with no ingress entry simply cannot be snapshotted from the control plane, and the API says so rather than failing obscurely. This is deliberately per-process configuration rather than database or directory data: keeping cluster ingress out of the broadcast directory is what stops an upstream adapter finding the cluster and skipping the gateway.

Scheduler

Variable	Default
CONTROL_SCHEDULER_ENABLED	true
CONTROL_SCHEDULER_INTERVAL_MS	5000

4.10 Authoring configuration in the control plane

The database holds a **draft topology** that nothing reads until it is published.

Table	Holds
shard	Shard id and the gateway client endpoints published in the directory
security	Identity and geometry (published), plus reference price and collar widths (sent as commands, never published)
participant	Participant id, name, SMP id, enabled
gateway, gateway_participant	Gateway identity, secret digest, and participant claims
spec_release, spec_release_shard	Published releases and the fingerprints frozen into each
session_schedule, session_schedule_entry, market_holiday	The trading calendar
control_user, operator_audit	Operators and who asked for every market-moving command

most

exchange control

Status

Operations

Books

Shards

Securities

Participants

Gateways

Releases

Schedules

Securities

Geometry — the floor, tick and ladder size — is published in the shard's security file and read at boot. Reference price and collars are not: they arrive at runtime as SecurityDefinition commands through the replicated log, so what is shown here is what an operator seeded, not what the engine currently holds. Changing either only edits the draft; geometry reaches a node through a published release and a restart, collars through [Operations](#).

New security

Id	Symbol	ISIN	Name	Ccy	Shard	Floor	Tick	Levels	Max orders	Reference	Static bps	Dynamic bps
1	AAPL	US0378331005	Apple Inc.	USD	0	0.00	0.01	32,768	1,000,000	—	—	—
2	MSFT	US5949181045	Microsoft Corporation	USD	0	0.00	0.01	32,768	1,000,000	—	—	—

Figure 4.1 Securities. Identity and ladder geometry, validated by the same domain objects a process uses at boot — the ISIN check and the length limits have exactly one implementation.

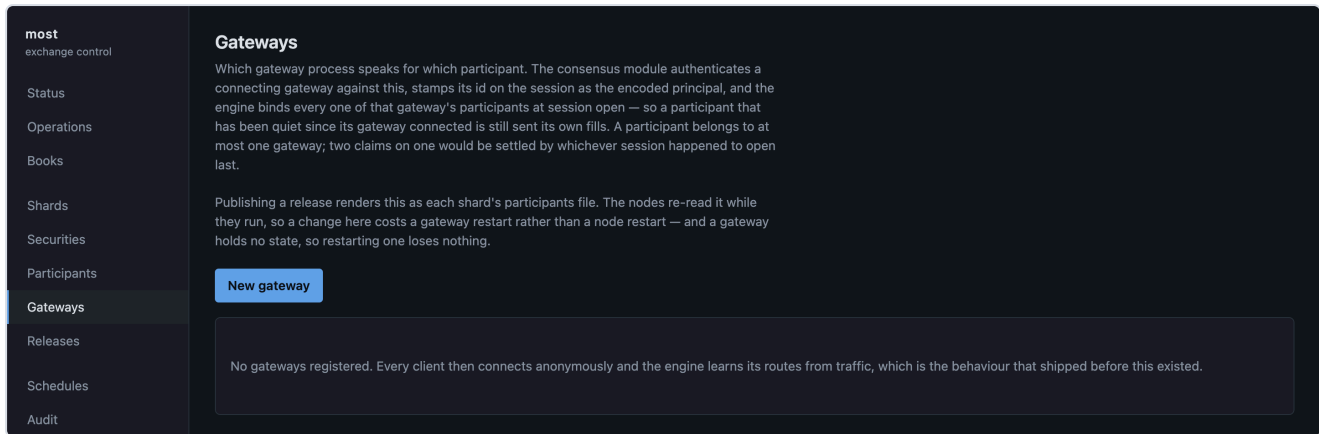


Figure 4.2 Gateways. Identity, participant claims, and the rendered registry a shard would publish. Creating a gateway returns its secret exactly once.

A GATEWAY SECRET IS RETURNED EXACTLY ONCE

`POST /api/gateways` generates one and returns the plaintext in that response and nowhere else; the database stores only its SHA-256. The digest cannot be BCrypt, because the cluster must reproduce it to verify what a gateway presents. Rotating with `PUT /api/gateways/{id}/secret` issues a new one — there is no way to read the old one back.

`GET /api/topology` reports **every** problem rather than the first, because a half-built topology is the normal state while someone is editing it. An empty `problems` list is exactly the condition for publishing, so one call answers both “can I publish?” and “what is stopping me?”.

4.11 Publishing a release

```
curl -u admin:... -X POST localhost:8080/api/releases \
  -H 'Content-Type: application/json' -d '{"note":"add TSLA to shard 1"}'
```

A release directory contains the rendered artifacts every process boots from, plus a manifest recording each shard’s geometry fingerprint and registry fingerprint separately.

most

exchange control

Status

Operations

Books

Shards

Securities

Participants

Gateways

Releases

Schedules

Audit

Operators

Releases

A release is an immutable numbered set of the same properties files the engine, gateway, market-data and discovery processes boot from. Republishing allocates the next version rather than editing one, so a directory a node booted from cannot change underneath it. The fingerprint is the value every process prints at startup; two nodes showing different ones have different geometry, which is the one misconfiguration consensus cannot catch.

Publish

Note (optional) — what changed, for whoever reads this at 3am

Publish release

Draft universe version 9181280125937456696.

Import a shard security file

The bootstrap path. Parsing goes through the same ShardSpec.f from a process boots with, so an imported file is validated by the code that would have booted it — and retying a tick size into a form is exactly where the typo this module exists to prevent gets in. The shard itself must exist first: endpoints are not in a security file.

```
shard.id=0
securities=...
```

Import

Version	Published	Universe	Directory	Fingerprints	Note	Files
9	06/09/2026, 16:39:25	9181280125937456696	/releases/000009	shard 0: 3e04cf2b9902f08a	dev stack	manifest discovery shard-0
8	05/09/2026, 18:38:08	9181280125937456696	/releases/000008	shard 0: 3e04cf2b9902f08a	dev stack	manifest discovery shard-0
7	05/09/2026, 18:32:29	9181280125937456696	/releases/000007	shard 0: 3e04cf2b9902f08a	dev stack	manifest discovery shard-0
6	05/09/2026, 18:14:19	9181280125937456696	/releases/000006	shard 0: 3e04cf2b9902f08a	dev stack	manifest discovery shard-0

Figure 4.3 Releases. Immutable and numbered; the exact bytes of any artifact can be read back, and an existing shard security file can be imported to seed the draft.

RELEASES ARE IMMUTABLE

Republishing allocates the next version rather than overwriting. A release carries **topology only** — Aeron directories, cluster directories and feed channels stay in each process’s own configuration.

Deploying a release is: fetch it to every machine, move the `current` symlink, and restart what needs restarting. What needs restarting depends on what changed — see 6.6.

⚠ PROCESSES DO NOT VERIFY THEY READ THE RELEASE YOU THINK THEY DID

Every process prints its fingerprint and the control plane records the fingerprint it published, but comparing them is the operator’s job. Making the four processes fail fast on a mismatch changes four boot paths and is deliberately a separate change.

5. Daily operations, order flow and monitoring

5.1 The trading day

Time	Action	How
Before the open	Purge expired orders	PurgeExpiredOrders, its own sequenced command
Before the open	Seed or re-seed reference prices and collars if they have moved	SecurityDefinition per security
Open – 1 hour	PRE_OPEN — orders accepted and booked, nothing matches	SessionTransition
Open – 5 minutes	OPEN_AUCTION — still accepting, still not matching	SessionTransition
Open	CONTINUOUS — the uncross runs on this transition	SessionTransition
Close	CLOSED — orders rejected, the resting book stays	SessionTransition
After the close	Take a snapshot	Cluster admin request

All five commands can be issued from the CLI, from the REST API, from the admin console, or by the control plane's scheduler. In production the scheduler does all of it and an operator intervenes.

EXPIRY IS DATE-BASED ONLY

Orders carry `expireDate` as a `YYYYMMDD` integer, where `0` means good-till-cancelled, and are expired when `0 < expireDate < currentTradingDate`. There is no intraday expiry. The purge is its own sequenced command and must run **well before** `PRE_OPEN`: after `PRE_OPEN` the book is accepting orders and a late sweep would expire orders someone had just placed. The scheduler skips a late purge rather than running it.

5.2 The trading calendar

A **schedule** is a named trading day — local times mapped to phases, plus a purge time — that a shard follows. It lives in the control plane and fires there, not in the engine.

That costs no determinism: the `SessionTransition` it emits is sequenced through the replicated log like any other command, so every node applies it at the same log position. What it buys is that holidays and daylight saving stay outside the deterministic state machine, where a bug kills every node simultaneously.

```
curl -X PUT localhost:8080/api/schedules/equities -u admin:... \
-H 'Content-Type: application/json' -d '{
  "name": "equities", "zone": "Europe/Amsterdam",
  "weekdays": ["MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY"],
  "purgeTime": "07:00",
  "entries": [
    { "at": "08:00", "phase": "pre-open" },
    { "at": "08:55", "phase": "open-auction" },
    { "at": "09:00", "phase": "continuous" },
    { "at": "17:30", "phase": "closed" } ] }'
```

```
curl -X PUT localhost:8080/api/shards/0/schedule -u admin:... \
-H 'Content-Type: application/json' -d '{"scheduleName":"equities"}'
```

```
curl -X POST localhost:8080/api/schedules/equities/holidays -u admin:... \
-H 'Content-Type: application/json' -d '{"date":"2026-12-25","description":"Christmas"}'
```

most

exchange control

Status

Operations

Books

Shards

Securities

Participants

Gateways

Releases

Schedules

Audit

Operators

admin ADMIN

Schedules

Every tick **reconciles**: the scheduler compares the phase the calendar wants against the phase the L3 feed reports and sends the difference, which makes it idempotent and self-healing after an outage. Two rules it keeps and an operator should know — the difference between phases is a path, not a destination, so catching up walks the intermediate phases (the uncross runs only on OPEN_AUCTION → CONTINUOUS); and a halted security is never reconciled back open, because that too would skip the auction. Recovery is **operator-driven**.

New schedule

Reconcile now

A tick runs on its own every few seconds; this is for not waiting.

Name	Zone	Trading days	Day	Purge	Holidays	State	
xetra	Europe/Amsterdam	MON TUE WED THU FRI	08:00 PRE_OPEN 09:00 OPEN_AUCTION 09:05 CONTINUOUS 17:30 CLOSED	07:00	2026-12-25 ×	enabled	Edit Holiday Delete

Which shard runs which schedule

A shard on no schedule is never moved by the scheduler; every transition on it is manual.

Shard

Schedule

0

xetra

Scheduler runs

What it did, and what it deliberately did not. A skip is recorded when its *reason* changes rather than on every tick, or a halted security would write the same row every few seconds all weekend.

At	Shard	Action	Phase	Trading date	Sent	Confirmed	Detail
02/09/2026, 20:49:05	0	session	CLOSED	20260902	sent	yes	every book on shard 0 reported CLOSED

Figure 5.1 Schedules. The calendar, its holidays, which shard follows which schedule, and the reconciler's own log of what it did and what it refused to do.

Times are **local**, and the zone is stored with them: a market opens at 09:00 local whatever the offset is that week. Storing UTC instants would move the open by an hour twice a year.

Every tick is a reconciliation

The scheduler does not fire timers. On each tick it compares the phase the calendar says a shard should be in against the phase **the L3 feed says it is in**, and sends the difference. One choice, three consequences: it is idempotent, it recovers by itself after an outage, and there is no missed-timer state to keep anywhere.

THE DIFFERENCE BETWEEN TWO PHASES IS A PATH, NOT A DESTINATION

The uncross runs only on `OPEN_AUCTION → CONTINUOUS`. A control plane that was down through the open and catches up at 09:15 walks `PRE_OPEN → OPEN_AUCTION → CONTINUOUS`. Sending `CONTINUOUS` straight from `CLOSED` would be accepted by the engine and would **silently skip the auction**, leaving a crossed resting book crossed until an aggressor happened to arrive. Closing is a single step — nothing is computed on the way down.

What the scheduler refuses to do, and says so rather than doing quietly:

- **It never reopens a halted security.** Recovery is operator-driven by design, and reconciling back to `CONTINUOUS` would do it *without an auction*, since from the scheduler's point of view the shard is already past `OPEN_AUCTION`.
- **It waits rather than guessing an unknown phase.** See 3.9.
- **It skips a late purge** rather than running it.
- **It stands down** when a shard's books disagree on their phase, and when another control-plane instance holds its Postgres advisory lock. A duplicated transition is survivable; a duplicated purge is not something to leave to luck.

```
GET /api/scheduler/runs          what it did, and what it deliberately did not
POST /api/scheduler/tick        reconcile now instead of waiting
GET /api/schedules/{name}/preview?date=2026-12-25
```

A skip is recorded when its **reason changes**, not on every tick, so a halted security appears once rather than every five seconds all weekend.

5.3 Operator commands

Five commands reach a running exchange. `SecurityDefinition`, `SessionTransition`, `PurgeExpiredOrders` and `RequestBookImage` go to the **gateway's client channel** like any other message and are forwarded into the log untouched. A snapshot request is different: it is a cluster admin request and goes to cluster ingress.

OPERATOR COMMANDS ARE NOT ACKNOWLEDGED

The engine applies or rejects them without replying. A rejected `SecurityDefinition` increments a counter and says nothing else. A sender may therefore claim only that **bytes were sent** — never that a command was applied. The control plane separates `sent` from `confirmed`, and confirms a phase only by afterwards seeing `SessionChanged` on the L3 feed.

Seeding a definition

```
$ most define --symbol AAPL --reference 101.30 --static-collar 5000 --dynamic-collar 100
defined AAPL on shard 0: reference 101.30, static 5000bps, dynamic 100bps
```

STATIC AND DYNAMIC REFERENCE

`staticReference` is seeded here and is reset only by an **executing** uncross. Its collar gates order *acceptance*: an order outside the band is rejected with `PRICE_OUT_OF_BOUNDS`. `dynamicReference` is seeded the same way but tracks **every** trade, and its collar gates *execution* per price level. An uncross that produces no trade leaves both untouched.

Everything checkable is checked **before** the bytes go out, because nothing will report a rejection. The control plane refuses a definition whose static band falls outside the pre-allocated ladder:

```
a 5000bp static band around 300.00 spans 150.00..450.00, outside AAPL's ladder
0.00..327.67; the engine would reject it without replying
```

Geometry in a definition is never taken from the caller — `priceFloor`, `tickSize` and `levelCount` come from the same rows the shard booted from, because a definition whose geometry disagrees with the allocated book is rejected in silence.

Moving the session

```
$ most session --phase continuous --shard 0
shard 0 → continuous (trading date 20260906)
```

```
{ "command": "session CONTINUOUS shard 0", "sent": true, "confirmed": true,
  "detail": "every book on shard 0 reported CONTINUOUS" }
```

A SESSION TRANSITION IS SHARD-WIDE

The engine applies the phase to every book it hosts. There is no per-security session command, so moving one security moves every other book on that shard. The REST response says so every time, not only when it bites.

Purging expired orders

```
$ most purge --shard 0
shard 0 purged for trading date 20260906
```

The purge walks the **price ladders**, not the order id map — that map compacts its probe chain on removal, so iterator-based removal can silently skip entries.

Republishing books as an image

For a market-data process that restarted while the engine kept running, and therefore has no book and no way to learn one from a live feed:

```
$ most image --shard 0
shard 0 asked to republish its books
```

BOOK IMAGE

One message per **occupied ladder level** — bounded by `levelCount` rather than by a resting depth nobody has measured — bracketed by begin and end messages. It carries the current sequence number as a **baseline and consumes none**, so a subscriber must not count an image's `seqNum` as a gap. Installing an image publishes increments as well as a snapshot, because a subscriber that is already synchronised ignores snapshots.

A snapshot restore also sets this pending, so both triggers use one path: publication happens from background work once the book-event publication is *connected*, because both ask at the moment a subscriber is least likely to be listening.

5.4 Order flow

The order entry message

Field	Notes
participantId	The trading firm
clOrdId	Client order id, echoed on every report for this order
price	Fixed point, 8 implied decimals
qty	Fixed point, 8 implied decimals
smpId	Self-match prevention id; 0 means use participantId
securityId	Validated by the gateway against its shard's list
expireDate	YYYYMMDD, or 0 for good-till-cancelled
side	BUY or SELL
smpStrategy	CANCEL_AGGRESSOR (default) or CANCEL_RESTING

A cancel carries `participantId`, `origClOrdId`, a new `clOrdId`, the `exchangeOrderId` to cancel, `securityId` and `side`.

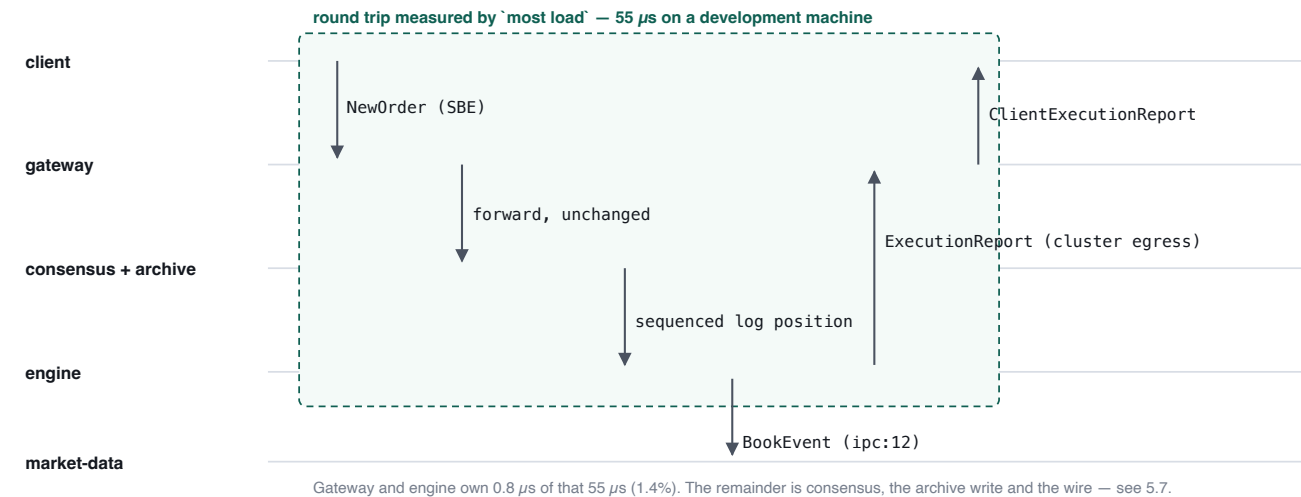


Figure 5.2 One client round trip through the stages that own it. The gateway and the engine are the two smallest.

What happens to an order

1. **Gateway.** `securityId` is validated against the shard's list. Anything else is rejected locally with `UNKNOWN_SECURITY` and never reaches the cluster. The message is then forwarded into cluster ingress **unchanged**.
2. **Consensus.** The message is replicated and sequenced. Every node will apply it at the same log position.
3. **Engine — admit.** Phase gate (`MARKET_CLOSED` outside the four phases below `CLOSED`), static collar (`PRICE_OUT_OF_BOUNDS`), ladder range (`PRICE_OUT_OF_LADDER`), capacity (`BOOK_CAPACITY`), expiry (`ORDER_EXPIRED`).
4. **Engine — match,** only in `CONTINUOUS`. Per price level the dynamic collar is checked first, as a level gate; self-match prevention is checked inside the level, as an order gate.
5. **Engine — settle.** The remainder is booked, execution reports are generated, and book events are published on the leader's IPC stream.

AN ORDER THE GATEWAY CANNOT FORWARD IS NOT CONSUMED

A cluster offer returning back-pressure is **transient**, and treating it as a drop would lose an order the client believes it placed, with no acknowledgement and no rejection. The gateway leaves the fragment unconsumed and offers the same bytes on the next poll, so congestion propagates back to the client's publication. A dead cluster session is not retryable and rejects with `GATEWAY_UNAVAILABLE`.

The **outbound** leg cannot do this — egress must keep being drained or the session dies — so it drops and counts. A rising `droppedToClient` means a subscriber is gone or too slow; the fix is a larger term buffer or a faster subscriber, not a busier gateway.

Rejection reasons

Reason	Raised by	Meaning
UNKNOWN_SECURITY	gateway, engine	securityId is not on this shard
MARKET_CLOSED	engine	The security is CLOSED. A volatility halt looks like this
PRICE_OUT_OF_BOUNDS	engine	Outside the static collar around staticReference
PRICE_OUT_OF_LADDER	engine	Outside the allocated ladder. Should be unreachable: the ladder is configured strictly wider than the band
BOOK_CAPACITY	engine	The order pool's high-water mark for this book
ORDER_EXPIRED	engine	expireDate is before the current trading date
UNKNOWN_ORDER	engine	A cancel for an order that is not resting
SELF_MATCH_PREVENTED	engine	The aggressor's remainder was cancelled by SMP
VOLATILITY_HALT	engine	The aggressor's remainder was cancelled by a dynamic collar breach
GATEWAY_UNAVAILABLE	gateway	The cluster session is closed
UNAUTHORIZED_PARTICIPANT	—	Defined on the wire and raised by nothing (1.7)

Self-match prevention

The effective SMP id is `smpId` if non-zero, otherwise `participantId`; it is resolved once at order entry and stored on the order. The **aggressor's** strategy governs.

Strategy	Behaviour
CANCEL_AGGRESSOR (default)	Stop matching, cancel the aggressor's remainder, leave the resting order
CANCEL_RESTING	Cancel the resting order and continue matching

A self-match never prints a trade, so it never moves either reference price.

SMP applies in the opening auction too, which makes the uncross a **fixed-point loop**: cancelling removes volume, which moves the uncross price, which crosses a different set of orders. An auction has no aggressor, so the later-arriving order of a self-matching pair plays that role and its strategy governs. The loop terminates because each iteration cancels at least one order from a finite book; `engine.auction.maxPasses` is a safety valve, not part of the algorithm.

The auction

The uncross runs on `OPEN_AUCTION` → `CONTINUOUS` and selects a price by, in order: maximum executable volume, then minimum imbalance, then the surplus side, then the price nearest the reference.

THE AUCTION IS UNCOLLARED

Neither collar constrains the uncross price. That is what lets a halted security reprice and reopen. It is not unbounded in practice, since every participating order passed the static collar when it was accepted.

A volatility halt

The dynamic collar is measured against a **snapshot** of `dynamicReference` taken when the aggressing order arrived — never the live value, which that order's own fills are advancing. On breach:

- the breaching fill is suppressed, and prior fills stand;
- the aggressor's remainder is cancelled with `VOLATILITY_HALT` and is not booked;
- the security moves to `CLOSED` with its resting book intact;
- a `VolatilityHalted` event is emitted on L3, and nowhere else.

There is no automatic recovery and no `VOLATILITY_HALT` phase. Recovery is 6.4.

5.5 Market data operations

Four feeds, on four channels, plus the recovery feed. A subscriber's responsibilities are fixed by the flow-control choice (3.7): detect gaps by sequence number, and re-synchronise from the snapshot feed.

FEED SEQUENCE

Every book event carries a `seqNum` and a `shardId`. The sequence is stamped by the engine where the event is generated, so it is deterministic across nodes and survives failover, and it is snapshotted alongside the next order id.

Sequences are namespaced by shard — each shard numbers from 1 independently — so several shards can share one multicast group and a subscriber tracks a sequence per shard.

Consumers should use `DepthFeedAssembler` and `FeedSequenceTracker` from the `reference` module. The operator CLI, the control plane's console and any FIX market data adapter all use the same two, so a book on one screen cannot disagree with a book on another.

A SYNCHRONISED SUBSCRIBER IGNORES SNAPSHOTS, WITH ONE EXCEPTION

Installing an image over an already-synchronised book would **rewind** it, because increments past that sequence were applied directly and never buffered. The exception is an image whose sequence is at or ahead of everything applied, which already contains all of it — without that exception, a subscriber handed a stale image on join discards every later one and can only be rescued by an increment happening to arrive. On a quiet book, none does.

An empty book still sends a bracketed zero-level cycle, because “no liquidity” and “I cannot yet know” are different answers.

5.6 Snapshots and restarts

NOTHING TAKES A SNAPSHOT UNLESS SOMETHING ASKS

Without one, a restart replays the log from genesis — which is precisely what the snapshot exists to avoid. Four things ask:

<code>most cluster snapshot -- ingress 0=HOST:PORT</code>	Through consensus, so every member snapshots at the same log position and the request is answered. This is the one to use in production.
<code>most cluster snapshot --dir DIR</code>	The local control toggle, one member only
<code>most cluster shutdown</code>	Snapshot, then stop — which plain SIGTERM does not do
The control plane's scheduler	At each session close

Leave the scheduler enabled in production: a daily snapshot at the close is what bounds restart time.

Restarting a node

```
most cluster shutdown --ingress 0=shard0-a:20110    # or rely on the scheduler's close snapshot
systemctl stop most-engine most-cluster-host
sleep 15                                           # mark-file liveness is ~10s; 15 for margin
systemctl start most-cluster-host most-engine
journalctl -u most-engine | grep 'restored'
```

That last line is the check. The engine prints `restored N resting orders ... from a snapshot` specifically so that a real recovery is distinguishable from the consensus module replaying the log into a freshly started service container.

RESTARTING ONLY THE SERVICE CONTAINER IS NOT A RECOVERY

If you stop the engine and leave the cluster host running, the consensus module replays the log to the new service from the beginning. It rebuilds the same books by a completely different route and takes as long as the session is old — and it says nothing about whether the snapshot works.

5.7 Measuring

`most load` drives a shard at a fixed rate and measures the round trip. The shard must already be defined and `CONTINUOUS`.

```
$ most load --symbol AAPL,MSFT --price-min 99.00 --price-max 103.00 \
  --rate 20000 --count 60000 --warmup 10000 --participant 7 --participants 2

load: sending 60,000 orders at 20,000/s (50 µs apart)
  1.0s sent 20,001 (20,000/s) reports 29,986 answered 19,941 in-flight 60 ack p99 7094.3
  2.0s sent 40,001 (20,000/s) reports 59,452 answered 39,902 in-flight 99 ack p99 6991.9
  3.0s sent 60,000 (19,966/s) reports 88,676 answered 60,000 in-flight 0 ack p99 6975.5

load: AAPL,MSFT 60,000 orders in 3.00s -- 20,000/s achieved (target 20,000/s)
  measured path client → gateway → cluster consensus → engine → gateway → client
  band          99.00..103.00 qty 1..100 participants 7..8 warmup 10000
  offers        ok=60,000 backpressure-retries=0 dropped=0
  reports       88,676 new=30,546 trade=19,080 canceled=9,596 expired=0 rejected=29,454
  fills         528,026 qty traded, 7,695 orders filled on arrival (12.8%)
  unanswered    0 orders never saw a report
  REJECTED      29,454 x MARKET_CLOSED
  pacing        lateness n=50,000 p50=0.0 p90=0.0 p99=0.1 p99.9=22.3 max=1132.5 (µs)
  ack response  n=50,000 p50=5275.6 p90=6537.2 p99=6975.5 p99.9=7622.7 max=8028.2 (µs)
  ack service   n=50,000 p50=5275.6 p90=6537.2 p99=6975.5 p99.9=7622.7 max=8028.2 (µs)
  fill service  n=7,695 p50=5238.8 p90=6512.6 p99=6983.7 p99.9=7721.0 max=7913.5 (µs)
```

Read it in this order:

1. **pacing lateness** **first**. If it is not small, the generator was the bottleneck and the run says nothing about the exchange. Its schedule is absolute — `start + i × delay` — because a relative sleep drifts and then catches up in bursts.
2. **The reject counts**. The run above is a worked example of a run that must be discarded: 29,454 `MARKET_CLOSED` rejections mean one of the two securities halted partway through, so half the orders never reached a book. Two things silently invalidate a run in exactly this way — a `maxOrders` too small for the rate, which turns everything into `BOOK_CAPACITY`, and a price band outside the static collar or the ladder. That is why the summary prints them.
3. **Both latencies**. *Service time* is measured from the actual send; *response time* from the scheduled send. Quoting only the first is coordinated omission and hides exactly the queueing that appears at the rate you are trying to find.

Run the generator on a separate machine, never on a cluster node.

ATTRIBUTION

`e2e/run-attribution.sh` splits a round trip by stage and writes `.hgrm` histograms meant to be diffed across a change. On a development machine the gateway and engine own **0.8 µs of a 55 µs** round trip — 1.4% — and the engine's whole-message p50 is 0.42 µs. The number to watch after a change is the *share*, not the round trip: real network hops add to the denominator and dedicated cores subtract from it.

5.8 Monitoring

What to watch

Signal	Where	Meaning
<code>droppedToClient</code>	gateway	Undeliverable execution reports. The outbound leg drops and counts by design; a rising rate means a subscriber is gone
<code>clusterBackpressure</code>	gateway	Transient and retried, so not a loss — but a sustained rate means ingress is saturated

Signal	Where	Meaning
untrackedReports	gateway	Reports the engine could not state an origQty for. Only orders restored from an older snapshot should produce these
gaps, eventsMissed	market-data	Feed sequence gaps. Occasional under MaxMulticastFlowControl; sustained means a subscriber cannot keep up
foreignShard	market-data	Book events for another shard. Means this process is pointed at the wrong engine
imagesApplied	market-data	A restarted process expects this to be non-zero. Zero with a book that stayed empty means the engine never sent one
rejectedDefinitions	engine	A SecurityDefinition was refused. Commands are unacknowledged, so this counter is the only signal
registryReloadFailures	engine, cluster-host	A registry that could not be read or was for another shard. The one in force still applies, so this is a quiet wrong rather than an outage
authenticatedGateways = 0	cluster-host, at shutdown	Every gateway connected anonymously despite a registry being configured
Leader changes	consensus	Any unexplained one is worth a look

⚠ THERE IS NO HEALTH OR METRICS ENDPOINT ON THE CORE PROCESSES

Everything above is a log line or a counter printed at shutdown. A scrape endpoint per process is what would make this mechanisable.

The exchange's own view of itself

`GET /api/status` is what the exchange is actually doing, as opposed to what the database says it should be. It is derived from L3 and from the discovery broadcast.

```
{
  "connected": true,
  "directory": { "version": "9181280125937456696", "securities": 2, "shards": [0],
    "lastSeenAt": "2026-09-06T14:44:00.827Z", "incompleteBroadcasts": 0 },
  "securities": [
    { "securityId": 1, "shardId": 0, "phase": "CLOSED",
      "halt": { "at": "2026-09-06T14:43:46.966Z", "collarReference": 10064000000,
        "attemptedPrice": 10167000000, "breachedBound": 100640000,
        "aggressorSide": "BUY", "clearedAt": null },
      "lastTradePrice": 10161000000, "lastTradeQty": 29 },
    { "securityId": 2, "shardId": 0, "phase": "CONTINUOUS", "halt": null,
      "lastTradePrice": 10122000000, "lastTradeQty": 4 }
  ],
  "feedGaps": 9543, "eventsMissed": 26075, "eventsSeen": 9552,
  "routingDrift": []
}
```

`routingDrift` reports where the database and what discovery is actually broadcasting disagree. Commands route from the **database**, not the directory — the control plane is the authority on topology and a command must still be sendable when discovery is down — so where the two disagree, that is said out loud rather than one silently winning.

Who asked

`operator_audit` records every market-moving command against the operator who issued it, and records `sent`, `not applied`. A command the control plane could not deliver is recorded too, with `sent: false`; that is exactly as interesting to an investigation as one that went out.

most

exchange control

Status

Operations

Books

Shards

Securities

Participants

Gateways

Releases

Schedules

Audit

Operators

Audit

Every market-moving command and who asked for it. **Sent is not applied**: operator commands are unacknowledged by design — the engine applies or rejects a definition, a session transition or a purge without replying — so a row saying **sent** claims only that the bytes left for the gateway. Unattended transitions are the scheduler's and are recorded in its own run log instead, with the reasons it skipped.

At	Operator	Action	Target	Sent	Detail
06/09/2026, 16:44:07	admin	reopen	shard:0	sent	define AAPL; session PRE_OPEN shard 0; session OPEN_AUCTION shard 0; session CONTINUOUS shard 0
06/09/2026, 16:39:27	admin	session	shard:0	sent	every book on shard 0 reported CONTINUOUS
06/09/2026, 16:39:26	admin	session	shard:0	sent	every book on shard 0 reported OPEN_AUCTION
06/09/2026, 16:39:25	admin	session	shard:0	sent	every book on shard 0 reported PRE_OPEN
06/09/2026, 16:39:25	admin	define	security:2	sent	seeded reference 100.00, static 5000bps, dynamic 2000bps -- not confirmable: no feed acknowledges a Sec
06/09/2026, 16:39:25	admin	define	security:1	sent	seeded reference 100.00, static 5000bps, dynamic 2000bps -- not confirmable: no feed acknowledges a Sec
05/09/2026, 18:44:36	admin	session	shard:0	sent	every book on shard 0 reported PRE_OPEN
05/09/2026, 18:42:22	admin	session	shard:0	sent	every book on shard 0 reported CLOSED
05/09/2026, 18:41:47	admin	session	shard:0	sent	every book on shard 0 reported CONTINUOUS
05/09/2026, 18:41:32	admin	session	shard:0	sent	every book on shard 0 reported OPEN_AUCTION
05/09/2026, 18:39:00	admin	session	shard:0	sent	every book on shard 0 reported PRE_OPEN
05/09/2026, 18:38:11	admin	session	shard:0	sent	every book on shard 0 reported CONTINUOUS
05/09/2026, 18:38:09	admin	session	shard:0	sent	every book on shard 0 reported OPEN_AUCTION
05/09/2026, 18:38:08	admin	session	shard:0	sent	every book on shard 0 reported PRE_OPEN
05/09/2026, 18:38:08	admin	define	security:2	sent	seeded reference 100.00, static 5000bps, dynamic 2000bps -- not confirmable: no feed acknowledges a Sec
05/09/2026, 18:38:08	admin	define	security:1	sent	seeded reference 100.00, static 5000bps, dynamic 2000bps -- not confirmable: no feed acknowledges a Sec
05/09/2026, 18:34:08	admin	session	shard:0	sent	every book on shard 0 reported OPEN_AUCTION
05/09/2026, 18:33:58	admin	session	shard:0	sent	every book on shard 0 reported PRE_OPEN
05/09/2026, 18:32:31	admin	session	shard:0	sent	every book on shard 0 reported CONTINUOUS

Figure 5.3 Audit. Every market-moving command, the operator who asked for it, and whether the bytes were sent.

Unattended transitions stay in the scheduler's own richer `scheduLe_run` log, which also records what it deliberately did *not* do.

Operator accounts

There is one role, `ADMIN`, with full access; the audit is what distinguishes who did what. Two ways in, for two different callers: a session cookie plus a CSRF token for the browser, and HTTP Basic — exempt from CSRF, because a request carrying its own credentials is not a forgery risk — for scripts.

most exchange control	Operators One role, full access. Every account here can create another account, move a session and reopen a halted security, so an account is not a permission boundary — the <code>audit</code> is what distinguishes who did what. A read-only tier is a data change rather than a migration (the role column exists), but nothing yet separates an observer from someone who can move a market.
Status	
Operations	
Books	
Shards	
Securities	
Participants	
Gateways	

New operator

Operator	Role	State	
admin	you	ADMIN	enabled
			Set password
			Disable
			Delete

Figure 5.4 Operators. Accounts and their state. The column exists so that adding a read-only tier later is a data change rather than a migration.

6. Troubleshooting and maintenance

6.1 First moves

Four questions answer most incidents, in this order.

1. **Do all processes agree on geometry?** Grep every startup log for `fingerprint`. One distinct shard value and one distinct registry value, or stop here.
2. **What phase does the exchange think it is in?** `GET /api/status`, which reads L3. Not the database, and not the calendar.
3. **Is the directory being broadcast?** `most securities`, or `directory.lastSeenAt` in the status response. A `most` command that hangs is almost always waiting for it.
4. **What do the counters say?** They are printed at shutdown, so a process that was `SIGKILL` ed tells you nothing. That is one of the reasons `SIGTERM` matters (3.8).

6.2 Startup failures

Symptom	Cause and remedy
missing required configuration key: <code>engine.securitiesFile</code>	No default exists. Give the process its properties file, or set the key as a system property
shard security file not found: ...	The current symlink points nowhere, or the release was never fetched to this machine
invalid ISIN 'US0378331006' for security 1	Check-digit validation at boot. The control plane refuses the same value with the same sentence
duplicate symbol in shard: [AAPL]	Two securities in one shard file share a symbol or an id
<code>engine.participantRegistry</code> is for shard 1, but this node serves shard 0	A registry from another shard's release. Caught here rather than at the first misrouted report
gateway id 'gw-x' is not in the participant registry	The gateway's id and the registry disagree. Stops the process rather than becoming a cluster connection the consensus module refuses with no explanation
<code>gateway.gatewayId</code> and one of ... must be set together	An id with no secret cannot authenticate; a secret with no id has nothing to authenticate as
no Aeron media driver found	The driver is not running, or <code>*.aeronDir</code> points somewhere else
could not reach the cluster	The consensus module is not running, or <code>gateway.ingressEndpoints</code> names the wrong hosts
Active media driver detected	A previous driver's liveness timeout has not expired, or the Aeron directory was preserved across a restart. It must not be — it is memory-mapped buffers and a <code>cnc.dat</code>
active mark file detected	A node restarted within roughly ten seconds of the previous one stopping. Wait
could not create archive directory	A volume or directory landed root-owned

Symptom	Cause and remedy
channel error - Cannot assign requested address	A channel names a host whose media driver does not own that address. In the dev stack that means anything other than shard0 (2.9)

6.3 The engine refuses to restore its snapshot

This is the system working. `loadSnapshot` reconciles the snapshot against the security file the node just booted from, and refuses rather than losing state.

```
matching-engine: refused to restore its snapshot -- security 2 (MSFT) is not in this shard's security file but holds 4 resting orders
```

Refusal	What to do
A security is gone from the file but holds resting orders	Restore the previous release, drain the book, then apply the change
priceFloor, tickSize, levelCount or maxOrders changed	Same — geometry cannot change under a populated book
An order falls outside the new ladder	Same
Resting-order counts do not add up	Do not restart around this. It means the snapshot and the file describe different things

The two legitimate cases are quiet and are logged with one line each: an **empty** book leaving the shard, and a new security joining.

A REFUSED RESTORE EXITS THROUGH THE SHUTDOWN BARRIER, NOT A HALT

Halting would skip the service container's close, which leaves the cluster mark file carrying a live timestamp — so the operator who fixes the file and restarts immediately would be met with `active mark file detected` for the next ten seconds instead of a working node.

A BAD ORDER DURING A RESTORE DOES NOT CRASH THE NODE

Aeron's `Image.poll` swallows an exception from its fragment handler, hands it to the client error handler, and advances the position anyway. The book would come back quietly wrong. This is why the restore range-checks every order itself and reconciles resting-order counts as a backstop — and why you should not assume a throw on this path would be loud.

6.4 Recovering a halted security

A halt sets **one security** to `CLOSED` and leaves the resting book intact. There is no halt phase, and `VolatilityHalted` is visible on L3 and nowhere else — a depth subscriber cannot tell a halt from a scheduled close.

Confirm it first:

```
"securities": [ { "securityId": 1, "phase": "CLOSED",
  "halt": { "at": "...", "collarReference": 10064000000, "attemptedPrice": 10167000000,
    "breachedBound": 100640000, "aggressorSide": "BUY", "clearedAt": null } } ]
```

Then reopen, in one operation:

```
curl -u admin:... -X POST localhost:8080/api/shards/0/reopen \
-H 'Content-Type: application/json' -d '{"securityId":1,"referencePrice":10161000000}'
```

```
{ "shardId": 0, "securities": ["AAPL", "MSFT"], "succeeded": true,
  "steps": [
    { "command": "define AAPL", "sent": true, "confirmed": false,
      "detail": "seeded reference 101.61 -- not confirmable: no feed acknowledges a
        SecurityDefinition, so a rejection would be silent" },
    { "command": "session PRE_OPEN shard 0", "sent": true, "confirmed": true },
    { "command": "session OPEN_AUCTION shard 0", "sent": true, "confirmed": true },
    { "command": "session CONTINUOUS shard 0", "sent": true, "confirmed": true } ],
  "warning": "a session transition is shard-wide: this moved AAPL, MSFT, not only AAPL" }
```

BOTH ORDERINGS ARE LOAD-BEARING

Re-seed the definition first. `staticReference` is reset only by an *executing* uncross, while orders are accepted from `PRE_OPEN` onward. If the halt moved price outside the old static band, the very orders needed to reopen are rejected by the stale collar before the auction that would have fixed it gets any — a deadlock that looks like nothing happening.

Walk the phases in full. The uncross runs only on `OPEN_AUCTION → CONTINUOUS`. Jumping straight to `CONTINUOUS` is accepted and silently skips the auction.

Doing it by hand is the same four steps:

```
most define --symbol AAPL --reference 101.61 --static-collar 5000 --dynamic-collar 100
most session --phase pre-open --shard 0
most session --phase open-auction --shard 0
most session --phase continuous --shard 0
```

The scheduler will not do this for you, by design (5.2).

6.5 Order flow problems

Symptom	Diagnosis
A client's orders are all MARKET_CLOSED	The security is CLOSED. Check /api/status for a halt block — a halt and a scheduled close are indistinguishable from a depth feed
A client's orders are all PRICE_OUT_OF_BOUNDS	The static collar is anchored on a stale staticReference. Re-seed the definition
Everything is BOOK_CAPACITY	maxOrders is too small for the rate. Under a load test this invalidates the run entirely
A maker never receives its fills	The participant is not bound to a live session. Either no registry is configured — so routes are learned from traffic only, and a participant that has been quiet since connect is unreachable — or its gateway connected anonymously
Duplicate orders	Two gateways subscribed to one client.inbound.channel. Give each its own (3.2)
Reports stop arriving but orders are accepted	droppedToClient is climbing: the outbound subscriber is gone or too slow
Orders are accepted but nothing matches	The phase is not CONTINUOUS. Booking without matching is correct in PRE_OPEN and OPEN_AUCTION

Symptom	Diagnosis
A crossed book that never uncrosses	CONTINUOUS was reached from somewhere other than OPEN_AUCTION, so the uncross never ran

ANONYMOUS GATEWAYS FAIL QUIETLY

A gateway that connected with no credentials trades perfectly well. It loses only the fills of whichever of its participants have gone quiet — invisible until someone reconciles a `cumQty`. The cluster host prints its authenticated-gateway count at shutdown for exactly this reason.

6.6 Market data problems

Symptom	Diagnosis
A subscriber shows nothing and says it is waiting	It has not received a snapshot yet. A full pass is <code>md.snapshot.cycleMs × securities</code>
A subscriber shows nothing after the engine restarted	It restarted too, and has no book. Request a book image (5.3)
gaps climbing steadily	A subscriber cannot keep up. Expected occasionally under <code>MaxMulticastFlowControl</code> ; sustained means a real problem, and the fix is not changing flow control
The feed stops minutes after everything looked fine	No IGMP querier on the VLAN, so snooping switches aged out the group
<code>foreignShard</code> non-zero	This market-data process is subscribed to another shard's engine
A halt is invisible	It is only on L3. Subscribe to L3, not L1 or L2
<code>imagesDiscarded</code> non-zero	Images arrived at an already-synchronised book that was ahead of them. Expected on join; sustained means something is republishing stale images

6.7 Maintenance procedures

Changing geometry

Geometry changes are applied by restarting, and the snapshot reconciliation (6.3) is what makes that safe.

1. Author the change in the control plane and publish a release.
2. Drain, or confirm empty, every book whose geometry changed or which is leaving the shard.
3. Fetch the release to every machine and move ``current``.
4. Restart the cluster host and engine on each node in turn, waiting out the mark-file timeout.
5. Restart gateway, market-data and discovery so they read the new file too.
6. Compare fingerprints.

Changing who speaks for whom

A participant list change does **not** restart a node.

```
# 1. Publish from the control plane, then point `current` at it on every machine.
ansible shard0,gateways -a "ln -sf /etc/most/releases/000042 /etc/most/current"

# 2. Watch both node processes adopt it. No restart.
journalctl -u most-engine -f | grep -m1 'registry: reloaded'

# 3. Restart the gateways so their sessions re-open and re-derive their bindings.
ansible gateways -a "systemctl restart most-gateway"
```

Check `registryReloadFailures` rather than assuming a silent success — a bad file is ignored, not fatal (4.3). `engine.participantRegistry.reloadMs=0` and `--participants-reload-ms 0` turn the poll off if a deployment would rather restart.

Replacing or adding a gateway

Gateways hold no state, so this is ordinary:

1. Create the gateway in the control plane and record the secret it returns **once**.
2. Publish a release, and deploy the registry to the cluster nodes.
3. Give the new gateway its own client inbound and outbound endpoints — never a peer's.
4. Start it, and point its adapters at it.

Removing one is the reverse; its adapters reconnect elsewhere and its in-flight orders are reported correctly by whichever gateway the client next uses.

Rotating a gateway secret

`PUT /api/gateways/{id}/secret` issues a new one and returns the plaintext once. Publish, deploy the registry, and restart that gateway. The nodes pick the new digest up without restarting, so the window in which the old secret still works is one poll interval.

Upgrading a node

Rolling, one member at a time, waiting for the group to be healthy between each:

```
most cluster snapshot --ingress 0=shard0-a:20110 # every member snapshots at the same position
systemctl stop most-engine most-cluster-host # on one node only
# deploy the new binaries
sleep 15
systemctl start most-cluster-host most-engine
journalctl -u most-engine | grep 'restored'
```

A WIRE CHANGE IS NOT A ROLLING UPGRADE

Nodes running different schema versions over one log will diverge. Treat any change to `message-schema.xml` as requiring a coordinated restart of the whole shard, and re-run `e2e/run-e2e.sh` first — it exists because unit tests cannot see the wire.

Checking a native image you cannot run

The native build needs `--add-exports=java.base/jdk.internal.misc=ALL-UNNAMED`. A missing export does **not** fail the build: it silently omits the class, and the binary dies on its first `UnsafeBuffer` — and only once a real Aeron control file exists, so a smoke test against a *missing* driver passes and hides it.

```
grep -c 'jdk.internal.misc.Unsafe' /opt/most/bin/engine # 0 = broken, 3 = correct
```

Starting the binary without a media driver does not check this: it exits on the expected driver timeout before the first buffer is ever wrapped.

6.8 Reading the logs

Line	Process	Means
cluster: started, awaiting shutdown signal	cluster-host	The consensus module is up. The engine may now attach
matching-engine: started, awaiting shutdown signal	engine	The service container is attached
matching-engine: restored N resting orders ... from a snapshot	engine	A real recovery, as distinct from a log replay
gateway: identity=gw-0 registry=... participants=[7, 8]	gateway	Authenticated, and speaking for those participants
gateway: no cluster identity	gateway	Connecting anonymously. Routes will be learned from traffic only
gateway: cluster session closed; orders can no longer be forwarded	gateway	Terminal for this process
cluster: registry: reloaded	cluster-host, engine	A new participant registry came into force, with both fingerprints
discovery: version=... shards=[0] securities=2	discovery	The universe version now being broadcast

Shutdown lines carry the counters, which is why an orderly SIGTERM matters:

```
gateway: stopped. forwardedToCluster=60000 clusterBackpressure=0 sentToClient=88676
droppedToClient=0 rejectedLocally=0 unreachableRejects=0
undeliverableCommands=0 untrackedReports=0

market-data: stopped. gaps=0 missed=0 foreignShard=0 droppedL1=0 droppedL2=0 droppedL3=0
snapshots=929 droppedSnapshot=0 imagesApplied=1 imagesDiscarded=0 imageOutOfBand=0
```

6.9 Consolidated list of what is not implemented

Area	What is missing	Section
Order entry	UNAUTHORIZED_PARTICIPANT is defined on the wire and raised by nothing, so a gateway may trade for a participant it does not claim	1.7
Cluster host	Member id is hardcoded to 0; no --member-id	3.6
Cluster host	Driver and archive threading modes are hardcoded SHARED	3.4
All processes	Idle strategies are constants, not configuration	3.4
Build	engine.march is x86-only	3.10
Discovery	One order-entry channel per shard, so several gateways cannot be advertised individually	3.2
Market data	A feed is one channel; multicast and dynamic MDC cannot coexist	3.7

Area	What is missing	Section
All processes	No health or metrics endpoint; monitoring is log lines and shutdown counters	5.8
Boot	Processes do not verify the release fingerprint they read against what was published	4.11
Engine	--gc=epsilon is proven by three measurements but is off by default, pending a CI to run the assertion in and a soak measured in hours	—
Production	None of section 3 has been run as described	3.10

7. Index

References are to section numbers. **Bold** marks where a term is defined.

7.1 Terms and concepts

A–C

Aeron directory 2.9, 4.8, 6.2
Anonymous gateway 4.3, 6.5, 6.8
Archive 3.3, 3.6, 4.8, 5.6
Auction, opening 1.6, 5.4
Auction, uncollared 5.4
Back-pressure, cluster ingress 5.4, 5.8
Book capacity 4.2, 5.4, 6.5
Book event stream 1.2, 3.2, 4.4, 4.6
Book image 5.3, 6.6
Circuit breaker, see Collar
Collar, dynamic 5.3, 5.4
Collar, static 4.2, 5.3, 5.4, 6.5
Conflation, depth images 2.8, 4.9
Consensus module 1.2, 3.6, 4.8
Coordinated omission 5.7
Core isolation 3.4
CSRF 2.8, 5.8
cumQty 2.6, 4.5, 6.5

D–G

Determinism 1.1, 4.4, 5.2
Directory, see Discovery
Discovery broadcast 1.2, 2.5, 4.7, 5.8
Dynamic MDC 2.9, 3.7
Egress, cluster 4.5, 5.4
Expiry, order 5.1, 5.3
Feed sequence 5.5, 6.6
Fingerprint, registry 2.4, 4.3
Fingerprint, shard geometry 1.4, 2.4, 3.9, 4.11, 6.1
Fixed point, 8 decimals 1.5, 4.2
Flow control, multicast 3.7, 6.6
Gateway identity 4.3, 4.5, 6.7
Geometry, ladder 4.2, 6.3, 6.7

H–O

Halt, volatility 1.6, 5.4, 6.4
Huge pages 3.3
IGMP querier 3.7, 6.6

Ingress, cluster 3.6, 4.5, 4.9
ISIN validation 4.2, 6.2
L1 / L2 / L3 1.2, 4.6, 5.5
Ladder, price 4.2, 5.4
Leader-only publication 3.2
Mark file, liveness 2.12, 3.8, 5.6, 6.2
Media driver 1.2, 2.3, 2.9, 3.4
Member string 3.6, 4.8
Metrics, hot path 4.4, 4.5, 5.7
Multicast, market data 3.7, 4.6
origQty 2.6, 4.5, 5.8

P–S

Participant binding 4.3, 6.5, 6.7
Participant registry 4.3, 4.4, 4.5, 4.8, 6.7
Phase, session 1.6, 5.1, 5.2
Purge, expired orders 5.1, 5.3
Reject reasons 5.4
Release, immutable 1.4, 3.8, 4.11, 6.7
Reference price, static and dynamic 5.3, 5.4, 6.4
Routing drift 5.8
SBE 1.3, 6.7
Scheduler, session 5.2, 3.9, 6.4
Self-match prevention 5.4
sent vs confirmed 5.3, 5.8
Shard 1.1, 4.2
Shard security file 4.2, 4.4–4.7
Snapshot, cluster 5.6, 6.3, 6.7
Snapshot, depth recovery 4.6, 5.5, 6.6
Synchronised, depth subscriber 2.7, 5.5, 6.6

T–Z

Threading modes 3.4, 3.10
Tick size 1.5, 4.2
Topology, draft 4.10, 4.11
Trading date 5.1, 5.3
Uncross 1.6, 5.4, 6.4
Universe version 2.5, 5.8
Zero allocation 2.2, 6.9

7.2 Commands

Operator CLI (most)

`most securities` 2.5
`most send` 2.6, 5.4
`most cancel` 2.6
`most book` 2.7
`most load` 5.7
`most define` 5.3, 6.4
`most session` 5.3, 6.4
`most purge` 5.3
`most image` 5.3, 6.6
`most cluster` 2.10, 4.8
`most cluster snapshot` 5.6, 6.7
`most cluster shutdown` 5.6

Control plane REST

`POST /api/auth/login` 5.8
`GET /api/status` 5.8, 6.1, 6.4
`GET /api/topology` 4.10
`POST /api/import` 4.11
`POST /api/releases` 4.11
`GET /api/releases/{v}/files/{name}` 4.11
`POST /api/gateways` 4.10, 6.7
`PUT /api/gateways/{id}/secret` 4.10, 6.7

`GET /api/gateways/registry/{shardId}` 4.10
`POST /api/securities/{id}/definition` 5.3
`POST /api/shards/{id}/session` 5.3
`POST /api/shards/{id}/purge` 5.3
`POST /api/shards/{id}/reopen` 6.4
`POST /api/shards/{id}/book-image` 5.3
`POST /api/shards/{id}/snapshot` 5.6
`GET /api/books, /books/stream` 2.8
`GET /api/books/status` 6.6
`PUT /api/schedules/{name}` 5.2
`PUT /api/shards/{id}/schedule` 5.2
`GET /api/scheduler/runs` 5.2
`POST /api/scheduler/tick` 5.2
`GET /api/audit` 5.8

Build and test

`./gradlew build` 2.2
`./gradlew installDist` 2.2, 2.3
`./gradlew :sbe:generateSbeCodecs` 1.3
`e2e/run-e2e.sh` 2.11, 6.7
`e2e/run-restart.sh` 2.11
`e2e/run-attribution.sh` 2.11, 5.7
`e2e/run-epsilon-soak.sh` 2.11

7.3 Configuration keys and environment variables

Shard artifacts

`shard.id` 4.2, 4.3
`shard.securities` 4.2
`security.N.*` 4.2
`registry.gateways` 4.3
`gateway.<id>.secret` 4.3
`gateway.<id>.participants` 4.3

Engine

`engine.securitiesFile` 4.4, 6.2
`engine.participantRegistry` 4.4
`engine.participantRegistry.reloadMs` 4.4, 6.7
`engine.aeronDir` 4.4
`engine.clusterDir` 4.4
`engine.serviceId` 4.4
`engine.bookEvent.channel` / `.streamId` 4.4
`engine.auction.maxPasses` 4.4, 5.4
`engine.backpressure.alertThreshold` 4.4
`engine.metrics, .stages, .file` 4.4

Gateway

`gateway.securitiesFile` 4.5
`gateway.participantRegistry` 4.5
`gateway.gatewayId` 4.5, 6.2
`gateway.credentialToken` / `TokenFile` 4.5
`gateway.ingressChannel` 3.6, 4.5
`gateway.ingressEndpoints` 3.6, 4.5
`gateway.egressChannel` 4.5
`gateway.client.inbound.*` 4.5, 3.2
`gateway.client.outbound.*` 4.5
`gateway.metrics, .file` 4.5

Market data and discovery

`md.securitiesFile` 4.6
`md.bookEvent.*` 4.6
`md.l1.*, md.l2.*, md.l3.*` 3.7, 4.6
`md.snapshot.*` 4.6, 6.6
`discovery.shards` 4.7
`discovery.shard.N.*` 4.7

discovery.channel / .streamId 4.7
discovery.intervalMs 4.7

Cluster host flags

--dir, --aeron-dir 4.8
--archive-dir, --cluster-dir 4.8
--members, --host 3.6, 4.8
--participants 4.8
--participants-reload-ms 4.8, 6.7
--fresh / --keep 4.8

Control plane

CONTROL_DB_* 4.9
CONTROL_PORT 4.9
CONTROL_RELEASE_DIR 4.9
CONTROL_COOKIE_SECURE 2.9, 4.9
CONTROL_ADMIN_USER / _PASSWORD 4.9
CONTROL_AERON_ENABLED / _DIR 4.9

CONTROL_DISCOVERY_* 4.9
CONTROL_L1_* / L2 / L3 4.9
CONTROL_SNAPSHOT_* 4.9
CONTROL_DEPTH_PUBLISH_MS / _MAX_LEVELS 4.9
CONTROL_CLUSTER_INGRESS_<shardId> 4.9, 5.6
CONTROL_SCHEDULER_ENABLED / _INTERVAL_MS 4.9, 5.2

Aeron and JVM

aeron.term.buffer.length 3.5
aeron.ipc.term.buffer.length 3.5
aeron.socket.so_sndbuf / so_rcvbuf 3.5
aeron.rcv.initial.window.length 3.5
--add-opens java.base/jdk.internal.misc 2.1
--add-opens java.base/sun.nio.ch 2.1
--add-exports java.base/jdk.internal.misc 6.7
--install-exit-handlers 3.8
engine.march 3.10

7.4 Further reading

Document	Covers
docs/Design.md	The authoritative specification: determinism, the performance budget and memory footprint, the wire and the feeds, the native build
docs/ControlPlane.md	The control plane, its schema, releases, live control, authentication and scheduling
docs/ProdDeployment.md	The production tier in more detail than section 3
docs/LocalTesting.md	The manual walkthrough, and the benchmarking procedure
deploy/README.md	The Docker development stack and its network layout
web/README.md	The admin frontend